
数据结构与算法

目录

第 1 章 数据结构.....	2
1.1 数据结构基础.....	2
1.1.1 为什么要学习数据结构.....	2
1.1.2 数据结构是什么.....	4
1.1.3 数据结构和算法的关系.....	4
1.1.4 数据结构和编程语言的关系.....	5
1.1.5 内存的理解.....	5
1.1.6 数据结构的研究方向.....	6
1.2 线性结构之数组.....	11
1.2.1 基础知识.....	11
1.2.2 数组优缺点.....	12
1.2.3 功能定义.....	14
1.2.4 实现原理.....	14
1.2.5 代码实现.....	15
1.3 线性结构之链表.....	18
1.3.1 链表是什么.....	18
1.3.2 链表优缺点.....	20
1.3.3 功能定义.....	20
1.3.4 代码实现.....	21
1.4 线性结构之栈.....	25
1.4.1 栈的定义.....	25
1.4.2 相关概念.....	25
1.4.3 栈的使用举例.....	26
1.4.4 栈的存储结构.....	27
1.4.5 功能定义.....	27
1.4.6 代码实现.....	27
1.5 线性结构之队列.....	29
1.5.1 基本概念.....	29
1.5.2 队列的存储结构.....	30
1.5.3 队列的常见操作与问题.....	30
1.5.4 功能定义.....	31
1.5.5 代码实现.....	31
第 2 章 算法.....	35
2.1 算法基础.....	35
2.1.1 算法定义.....	35
2.1.2 算法评价标准 1：时间复杂度(效率的度量).....	36
2.1.3 算法评价标准 2：空间复杂度(存储量的度量).....	43
2.2 查找算法.....	47
2.2.1 顺序查找.....	47
2.2.2 二分查找.....	48
2.3 排序算法.....	50
2.3.1 十大常见的排序算法.....	50
2.3.2 冒泡排序.....	51
2.3.3 快速排序.....	53

第 1 章 数据结构

1.1 数据结构基础

1.1.1 为什么要学习数据结构



Linux内核之父林纳斯·托瓦兹 (Linus Torvalds)

我认为一个优秀的程序员和一个不合格的程序员之间的区别在于，他**是否将数据结构看得比代码更重要**。不合格的程序员总是在关注代码，而优秀的程序员则会更看重数据结构。



Pascal之父尼古拉斯·沃斯Nicklaus Wirth

提出了一个著名的公式：
算法 + 数据结构 = 程序

“Algorithms+Data Structures=Programs”

我们举一个形象的例子来理解程序中数据结构的作用：



战场：程序运行所需的软件、硬件环境。

敌人：项目或模块的功能需求。

指挥官：编写程序的程序员。

士兵和装备：一行一行的代码。

战术和策略：数据结构。



上图：没有战术，打仗事倍功半。



上图：有战术，打仗事半功倍。

由此可见，**数据结构和算法才是程序的核心**。但现实情况却是很多工作了三五年的程序员并不重视数据结构和算法，他们所实现的程序往往运行效率低且难以维护。

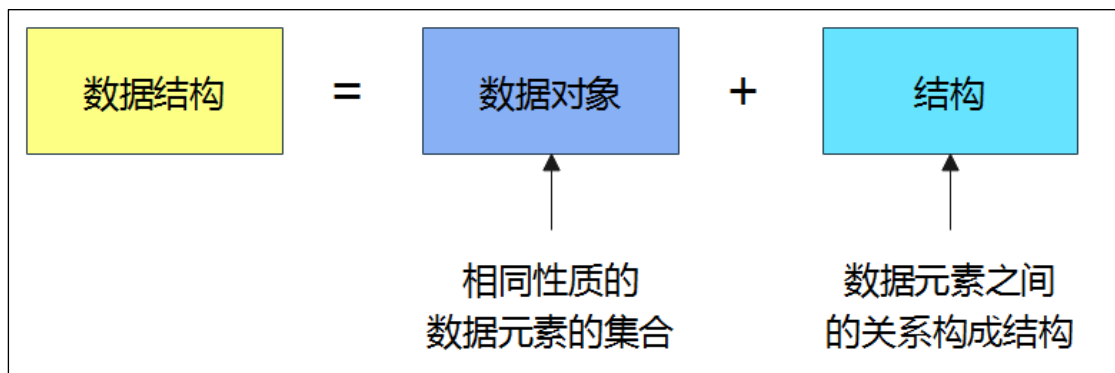
1.1.2 数据结构是什么

定义 1(宏观): 数据结构是为了**高效访问数据**而设计出的一种数据的组织和存储方式。

定义 2(微观): 数据结构是相互之间存在**一种或多种特定关系**的数据元素的集合。

简言之，数据结构是**带结构的数据元素的集合**，“结构”就是指数据元素之间存在的**关系**。

数据结构，关注的是对这些数据元素的操作和数据元素之间的**关系**，不关心具体的数据项内容。目的是为了**高效的访问数据**。

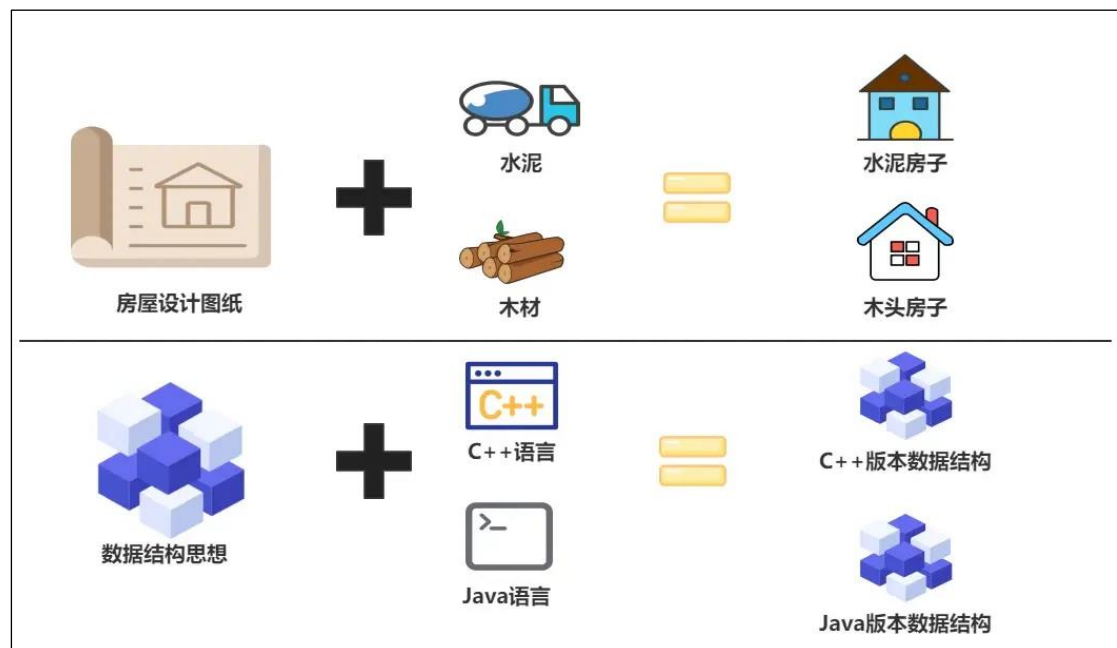


1.1.3 数据结构和算法的关系

数据结构是算法的基础，好的算法也离不开数据结构。

1.1.4 数据结构和编程语言的关系

简言之，**数据结构本身和编程语言是无关的**。二者关系类似于数学定理和描述它的自然语言一样，你可以用中文表达勾股定理，也可以用英文，法文或者世界上任何一种语言表达它，但是世上只有一个勾股定理。但如果任何一种语言你都不会，那就无法学好数据结构。还类似于房屋设计图纸和实现它所需要的水泥或木材一样。

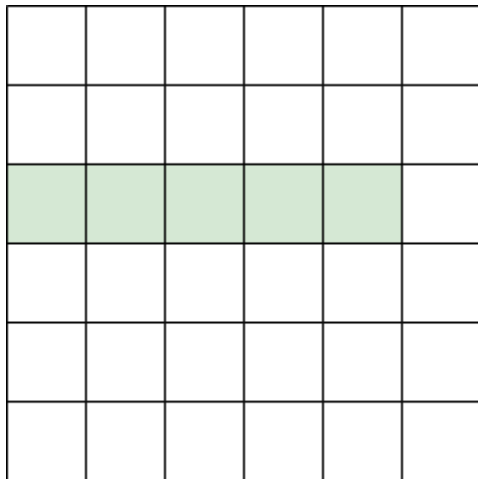


1.1.5 内存的理解

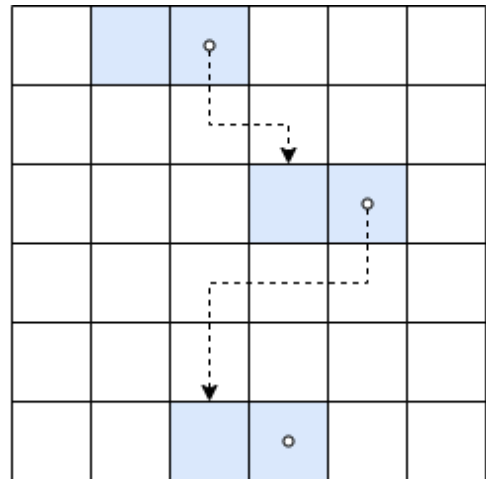
一般而言，数据结构针对的是内存中的数据。所以在学习数据结构之前，需要先对内存有一个简单的了解。

内存由许多存储单元组成，每个存储单元可以存储一个固定大小的数据块，通常以**字节（Byte）**为单位。每个存储单元都有一个**唯一的地址**，操作系统正是根据这一地址去访问内存中的数据。

我们讨论的数据结构中的数据元素就保存在这些一个个的内存单元中，这些数据元素或存储在**连续的**内存单元中，或存储在**分散的**内存单元中。



连续存储



分散存储

1.1.6 数据结构的研究方向

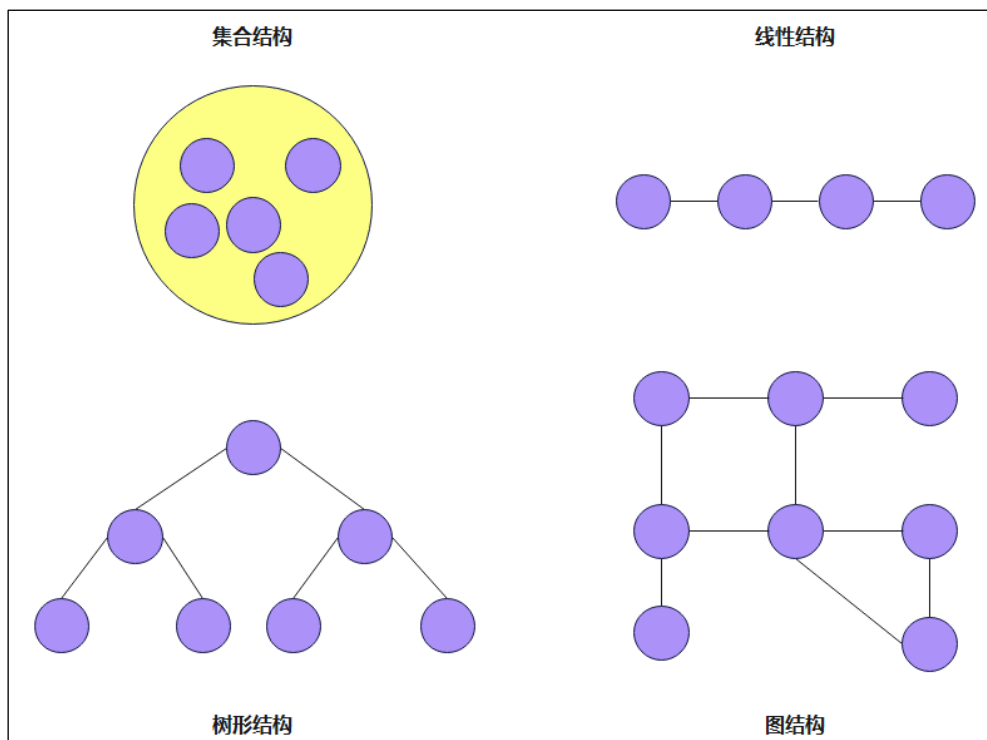
1.1.6.1 方向 1：数据的逻辑结构

数据元素之间逻辑关系的描述： $D, S = (D, S)$ 。这里有两个要素：数据元素、关系。其中，关系是指数据元素间的逻辑关系。

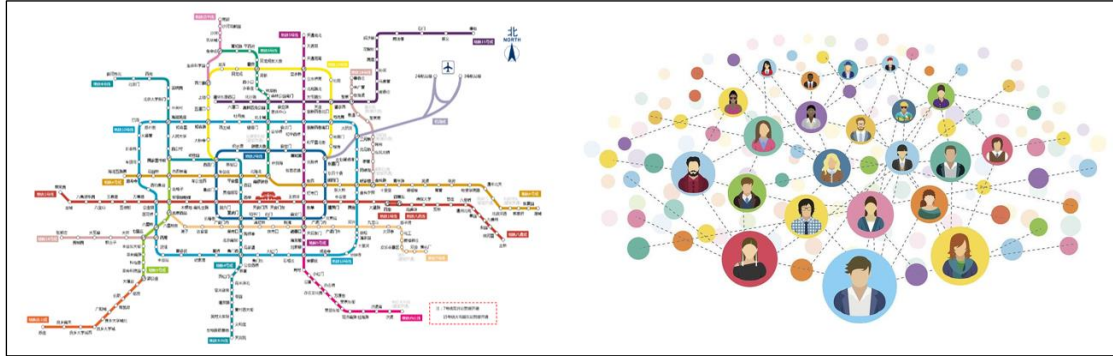
根据数据元素间关系的不同特性，可以分为：

1) 分类角度 1：

逻辑结构有四种基本类型：集合结构、线性结构、树形结构、图形结构（或网状结构）。



(4) **图形结构(网状结构)**: 数据结构中的元素存在多对多的相互关系。由顶点和边组成, 顶点对应数据元素, 边连接两个顶点, 表示两个数据元素之间的关系。比如: 全国铁路网、地铁图、社交网络。



2) 分类角度 2:

因为线性结构比较突出, 所以, 我们又习惯上将逻辑结构分为: **线性结构**和**非线性结构**。
其中:

- (1) 线性结构, 包括线性表(顺序表、链表)、栈、队列、字符串、数组、广义表等。
- (2) 非线性结构, 各个数据元素不再保持在一个线性序列中。比如, 集合结构、树结构, 图结构等。

3) 小结:

数据的逻辑结构指反映数据元素之间的逻辑关系, 是独立于计算机的, 与数据的**存储无关**(即与数据元素本身的形式、内容无关, 与数据元素的相对位置无关, 与所含结点数无关)。

比如, 栈属于线性结构, 是一种逻辑结构, 可以采用顺序存储, 也可以采用链式存储。

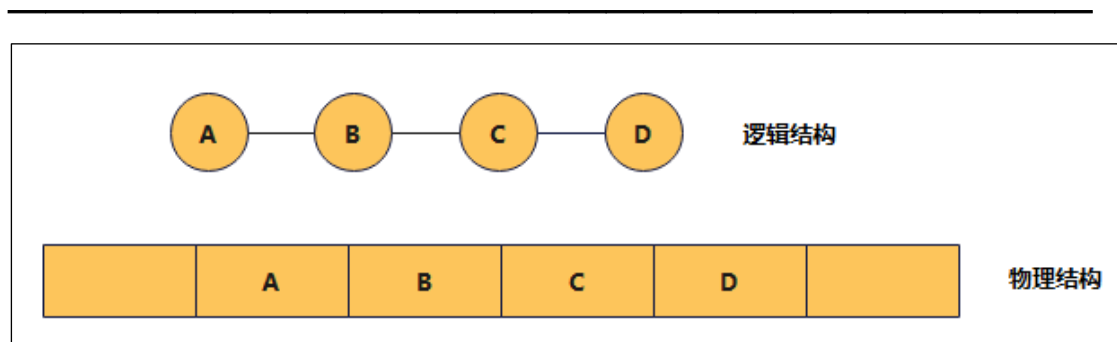
1.1.6.2 方向 2: 数据的存储结构

数据的存储结构又称为**物理结构**, 是数据的逻辑结构在计算机中的表示(又称映像)。它包括数据元素的表示和关系的表示(即逻辑关系)。

数据的存储结构是逻辑结构用计算机语言的实现, 它**依赖于计算机语言**。人们为了合理利用计算机的存储空间, 研究出了两种基本的存储方式:

1) 基本结构 1: 顺序存储结构

顺序存储结构把**逻辑上相邻**的数据元素存储在**物理位置上也相邻**的存储单元中, 即所有的元素依次存放在一片连续的存储空间中。



- 在 C 语言中，使用一维数组表示顺序存储结构。
- 优点：数据元素存放的地址是连续的，支持下标访问，随机存取表中元素。

2) 基本结构 2: 链式存储结构

链式存储结构把**逻辑上相邻**的数据元素存储在**物理上可以不相邻**的存储空间中。其中，为每个数据元素构造一个结点（用结构体类型表示），结点中除了存放数据本身以外，还存放指向下一个结点的指针(pointer)。

C 语言中用指针来表示数据元素之间的这种结构。

图 1:

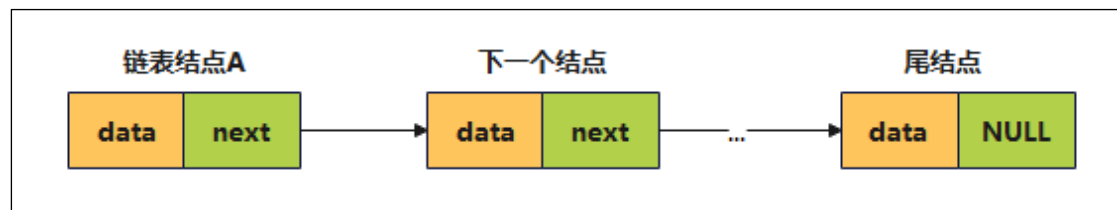
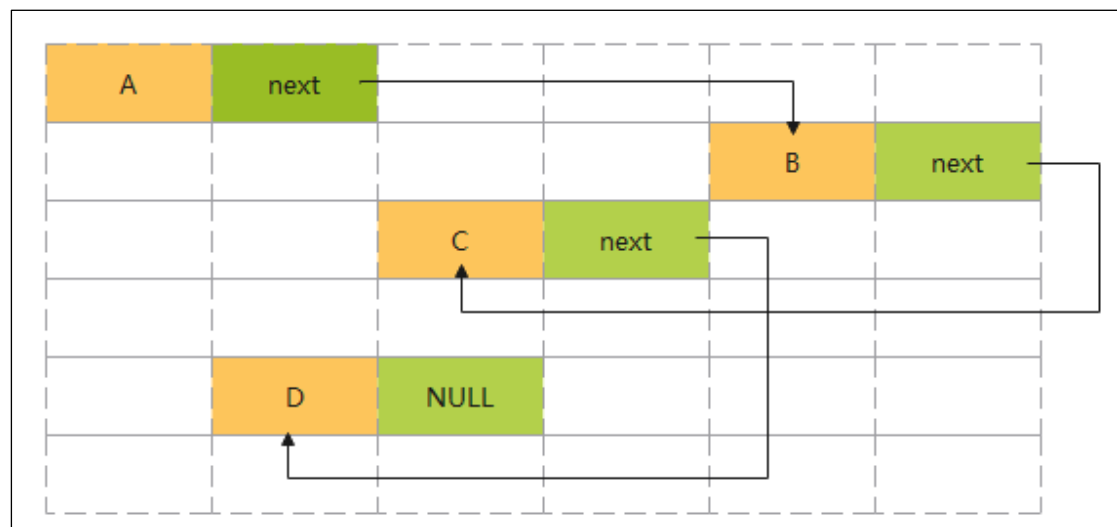


图 2:



- 优点：克服顺序存储结构中预知元素个数的缺点；插入、删除灵活（不必移动节点，只要改变节点中的指针）。

- 缺点：需要额外的空间来表达数据之间的逻辑关系（每个节点都由数据域和指针域组成，所以相同内存空间内全存满的话，顺序比链式存储更多数据）。

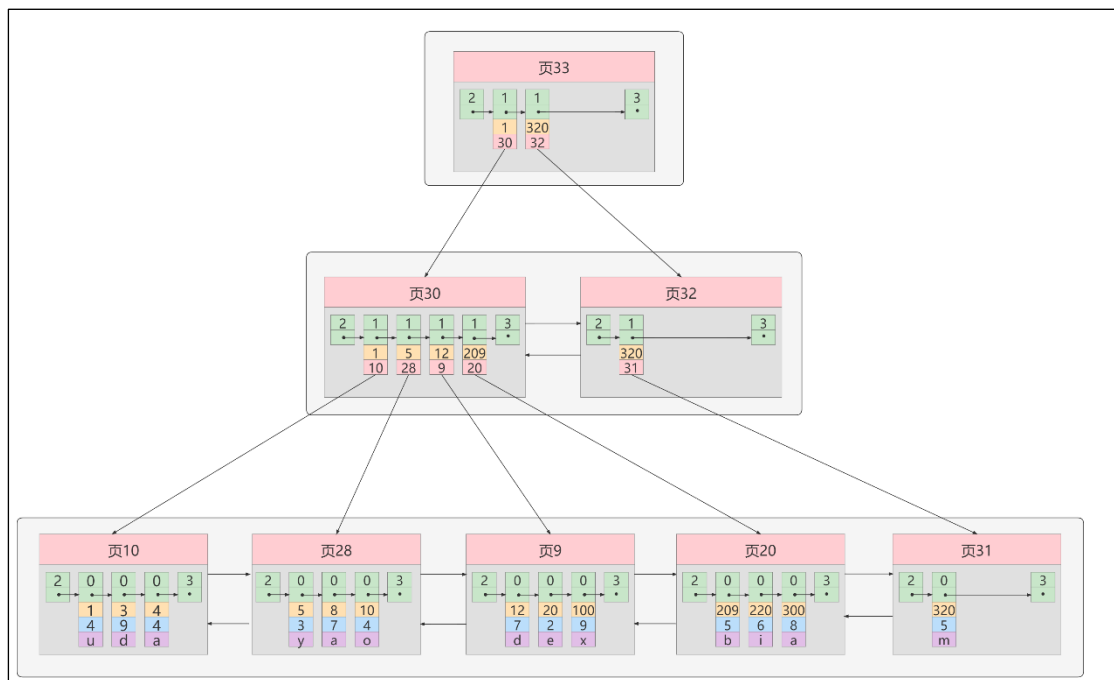
由两类基本的存储方式，又引申出两类存储结构：

3) 结构 3：索引存储结构

把索引想象成书的目录，有了目录，我们找书的相关内容就十分简单。索引存储在存储数据元素之外，同时建立数据元素的目录，便于快速检索。

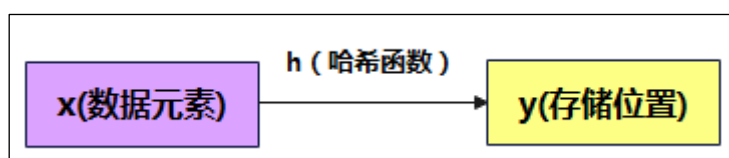
MySQL 数据库的索引的设计方案：

- 优点：用节点的索引号来确定结点存储地址，检索速度快。
- 缺点：增加了附加的索引表，会占用较多的存储空间。在增加和删除数据时要修改索引表，因而会花费较多的时间。

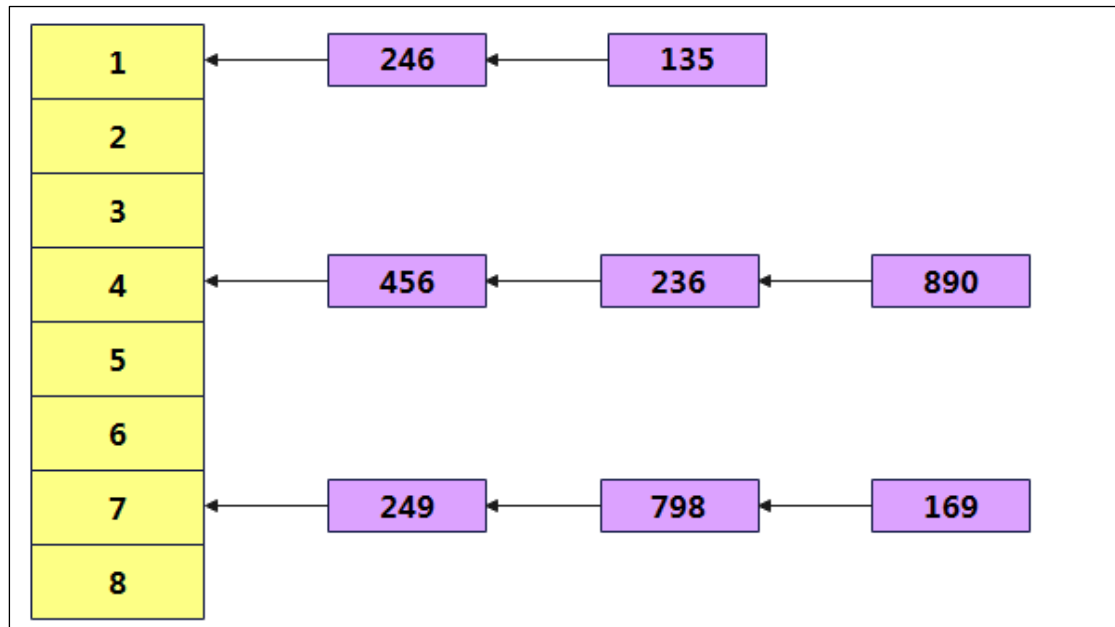


4) 结构 4：散列存储结构

根据数据元素的**关键字**直接计算出该元素的**关键码**，由关键码决定其存储地址，又称为**Hash 存储**。在初中，我们就学过一种能够将一个 x 值通过一个函数获得对应的一个 y 值的操作，叫做**映射**。散列表的实现原理正是映射的原理。在映射的过程中，事先设定的函数称作**散列函数或者哈希函数**。



- 优点：检索、增加或删除结点的操作都很快。
- 缺点：不支持排序，一般比用线性表存储需要更多的空间，并且记录的关键字不能重复。



1.1.6.3 方向 3：数据的运算

施加在数据上的运算包括运算的定义和实现。运算的实现是针对存储结构的，指出运算的具体操作步骤。

- 分配资源，建立结构，释放资源
- 插入和删除
- 获取和遍历
- 修改和排序

1.2 线性结构之数组

1.2.1 基础知识

数组是具有‘**相同类型**’的数据元素的集合。在所有的数据结构中，数组算是最常见、最简单的一种数据结构。

数组需要先声明才能使用，数组的长度一旦确定就‘**不可再变**’了。比如我们声明一个长度为 10 的数组：

```
int[] array = new int[10]; //java  
int array[10]; //c
```

数组的下标是‘**从零开始**’的，比如上面数组的第一个元素是 `array[0]`，最后一个元素是

array[9]。

index	0	1	2	3	4	5	6	7	8	9
value	7	6	2	9	4	5	3	1	8	6
array[0]=7, array[1]=6, array[2]=2, array[3]=9.....										

1.2.2 数组优缺点

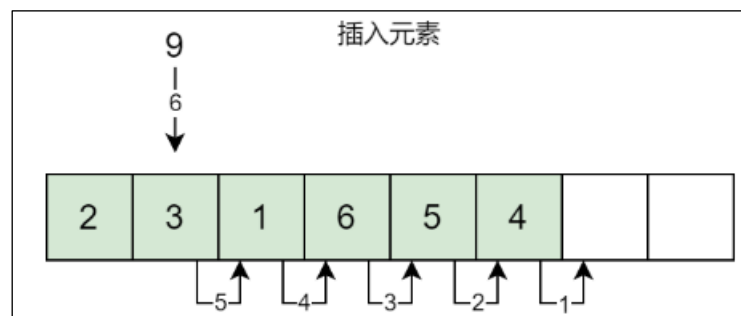
优点

- 查找容易（通过下标），时间复杂度为 $O(1)$ 。不需要额外申请或删除空间。
- 使用下标位置索引(index)十分高效的访问任意元素，修改快

数组下标	0	1	...	i-1	...	n-1
元素	a_1	a_2	...	a_i	...	a_n

缺点

- **插入、删除元素难，效率低。**（需要移动大量元素以使元素空间连续）。
- 插入操作平均需要移动 $n/2$ 个元素。
- 删除操作平均需要移动 $(n-1)/2$ 个元素。



- **扩展相对繁琐。**一方面需要确保能提供更大区域的连续内存空间，另一方面需要将原有数据复制到新的顺序表中。

练习 1：正序+反序打印数组

```
#include <stdio.h>
#define N 10

int main()
{
```

```

int arr[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

// 正向遍历
for (int i = 0; i < N; i++)
{
    printf("%d ", arr[i]);
}

printf("\n");

// 逆向遍历
for (int i = N - 1; i >= 0; i--)
{
    printf("%d ", arr[i]);
}

printf("\n");

return 0;
}

```

练习 2: 定义一个整型数组并赋初值, 遍历数组让每个索引位置的数组元素都除以首元素, 得到的结果作为该索引位置的新值。

```

#include <stdio.h>
#define N 7

int main()
{
    int arr[] = {12, 43, 65, 53, -38, 64, 2};

    //      for(int i = 0; i < N; i++){
    //          arr[i] = arr[i] / arr[0];
    //      }

    for (int i = N - 1; i >= 0; i--)
    {
        arr[i] = arr[i] / arr[0];
    }

    // 遍历 arr
    for (int i = 0; i < N; i++)
    {
        printf("%d ", arr[i]);
    }
}

```

```
printf("\n");

getchar();

return 0;
}
```

1.2.3 功能定义

前文提到过数组这一数据结构的一个局限性是长度固定，本节我们来实现一个增强版的数组——可变长的动态数组，需要实现以下函数：

//初始化动态数组
void initDynamicArray (DynamicArray *array, size_t initialCapacity)
//释放动态数组内存
void destroyDynamicArray (DynamicArray *array)
//调整动态数组内存大小
void resizeDynamicArray (DynamicArray *array, size_t newCapacity)
//获取动态数组长度（元素个数）
size_t getLength (const DynamicArray *array)
//在指定位置插入新元素
void insertAt (DynamicArray *array, size_t index, int element)
//在末尾插入新元素
void insertEnd (DynamicArray *array, int element)
//删除指定位置的元素并返回被删除的元素
int deleteAt (DynamicArray *array, size_t index)
//删除末尾的元素并返回被删除的元素
int deleteEnd (DynamicArray *array)
//遍历所有的元素
void print (DynamicArray *array)

1.2.4 实现原理

可变长的动态数组是一种数据结构，它允许在运行时根据需要动态地调整数组的大小，而不需要提前指定固定的大小。这种动态数组通常被称为动态数组、动态分配数组、动态增长数组或动态内存数组。int arr[10];

C 语言中是通过使用**指针**和**内存分配函数**来实现动态数组，常见的内存分配函数是 malloc、realloc 和 free。下面是一些相关的概念和操作：

（1）分配内存（malloc）：在 C 语言中，可以使用 malloc 函数来分配一块指定大小的内存。例如，int *arr = (int *)malloc(n * sizeof(int)); 将分配能够存储 n 个整数的内存空间。

(2) 重新分配内存 (realloc)： 如果需要改变动态数组的大小，可以使用 realloc 函数来重新分配内存。这允许你在保留原有数据的情况下扩展或缩小数组的大小。

(3) 释放内存 (free)： 当不再需要动态数组时，应使用 free 函数释放之前分配的内存，以避免内存泄露。→ 内存溢出

1.2.5 代码实现

```
#include <stdio.h>
#include <stdlib.h>

// 动态数组结构体
typedef struct
{
    int *data;          // 指向动态数组的指针
    size_t size;        // 当前数组中的元素个数
    size_t capacity;    // 当前数组的容量（可以容纳的最大元素个数）
} DynamicArray;

// 初始化动态数组
void initDynamicArray(DynamicArray *array, size_t initialCapacity)
{
    array->data = (int *)malloc(initialCapacity * sizeof(int)); // 分配初始内存
    array->size = 0;          // 初始化元素个数为 0
    array->capacity = initialCapacity; // 设置初始容量
}

// 释放动态数组内存
void destroyDynamicArray(DynamicArray *array)
{
    free(array->data); // 释放动态数组内存
    array->size = 0;    // 重置元素个数为 0
    array->capacity = 0; // 重置容量为 0
}

// 调整动态数组内存大小
void resizeDynamicArray(DynamicArray *array, size_t newCapacity)
{
    array->data = (int *)realloc(array->data, newCapacity * sizeof(int)); // 调整数组内存大小
    array->capacity = newCapacity; // 更新容量
}
```

```
// 获取动态数组长度（元素个数）
size_t getLength(const DynamicArray *array)
{
    return array->size; // 返回数组中的元素个数
}

// 在指定位置插入新元素
void insertAt(DynamicArray *array, size_t index, int element)
{
    if (index > array->size)
    {
        return; // 忽略无效的插入位置
    }

    if (array->size >= array->capacity)
    {
        size_t newCapacity = array->capacity * 2; // 如果容量不足，扩大容量
        resizeDynamicArray(array, newCapacity);
    }

    for (size_t i = array->size; i > index; i--)
    {
        array->data[i] = array->data[i - 1]; // 后移元素以腾出插入位置
    }

    array->data[index] = element; // 在指定位置插入新元素
    array->size++;                // 更新元素个数
}

// 在末尾插入新元素
void insertEnd(DynamicArray *array, int element)
{
    insertAt(array, array->size, element); // 在末尾插入新元素
}

// 删除指定位置的元素并返回被删除的元素
int deleteAt(DynamicArray *array, size_t index)
{
    if (index >= array->size)
    {
        return -1; // 忽略无效的删除位置
    }
}
```

```
int deletedElement = array->data[index]; // 获取被删除的元素

for (size_t i = index; i < array->size - 1; i++)
{
    array->data[i] = array->data[i + 1]; // 前移元素以填补删除位置
}

array->size--; // 更新元素个数

return deletedElement; // 返回被删除的元素
}

// 删除末尾的元素并返回被删除的元素
int deleteEnd(DynamicArray *array)
{
    return deleteAt(array, array->size - 1); // 删除末尾的元素
}

// 遍历所有的元素
void print(DynamicArray *array)
{
    for (int i = 0; i < array->size; i++)
    {
        printf("%d ", array->data[i]);
    }
    printf("\n");
}

int main()
{
    DynamicArray myArray; // 声明动态数组

    // 初始化动态数组
    initDynamicArray(&myArray, 2);
    printf("初始化动态数组, 初始容量为 2\n");

    // 向动态数组尾部插入元素
    insertEnd(&myArray, 1);
    insertEnd(&myArray, 2);
    printf("向动态数组尾部插入了 2 个元素\n");

    // 打印动态数组当前长度
    printf("动态数组当前长度:%zu\n", getLength(&myArray));
}
```

```

// 在索引 1 的位置插入元素 3
insertAt(&myArray, 1, 3);
printf("在索引 1 的位置插入元素 3\n");

// 再次打印动态数组当前长度
printf("动态数组当前长度:%zu\n", getLength(&myArray));

// 删除索引 1 的元素
printf("删除索引 1 的元素,该元素是%d\n", deleteAt(&myArray, 1));

// 删除动态数组末尾元素
printf("删除动态数组末尾元素,该元素是%d\n", deleteEnd(&myArray));

// 释放动态数组内存
destroyDynamicArray(&myArray);
printf("动态数组内存释放完成\n");

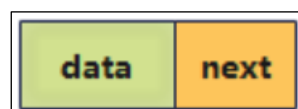
return 0;
}

```

1.3 线性结构之链表

1.3.1 链表是什么

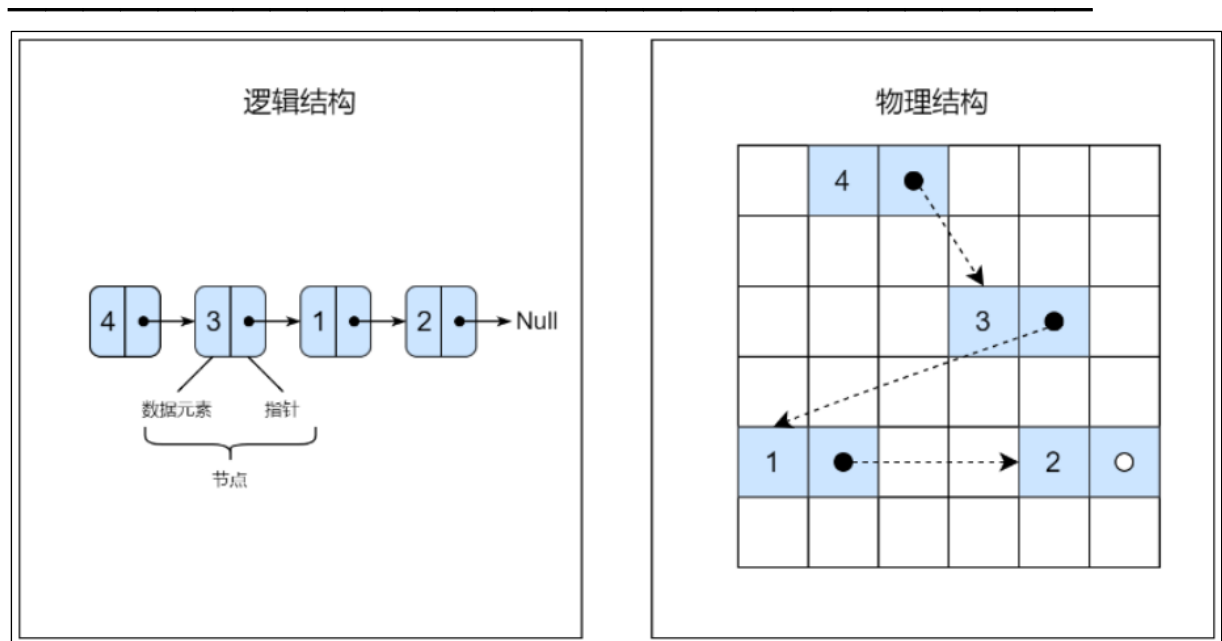
逻辑结构：链表是一个线性结构，由一系列‘**结点(Node)**’组成，每个结点包含一个数据元素和一个指向下一个结点的‘**指针(Pointer)**’。所有结点通过指针相连，形成一个链式结构。通常我们将链表中的第一个结点称为‘**头结点**’。



- data : 数据域，存放结点的值。
- next : 指针域，存放结点的直接后继的地址。

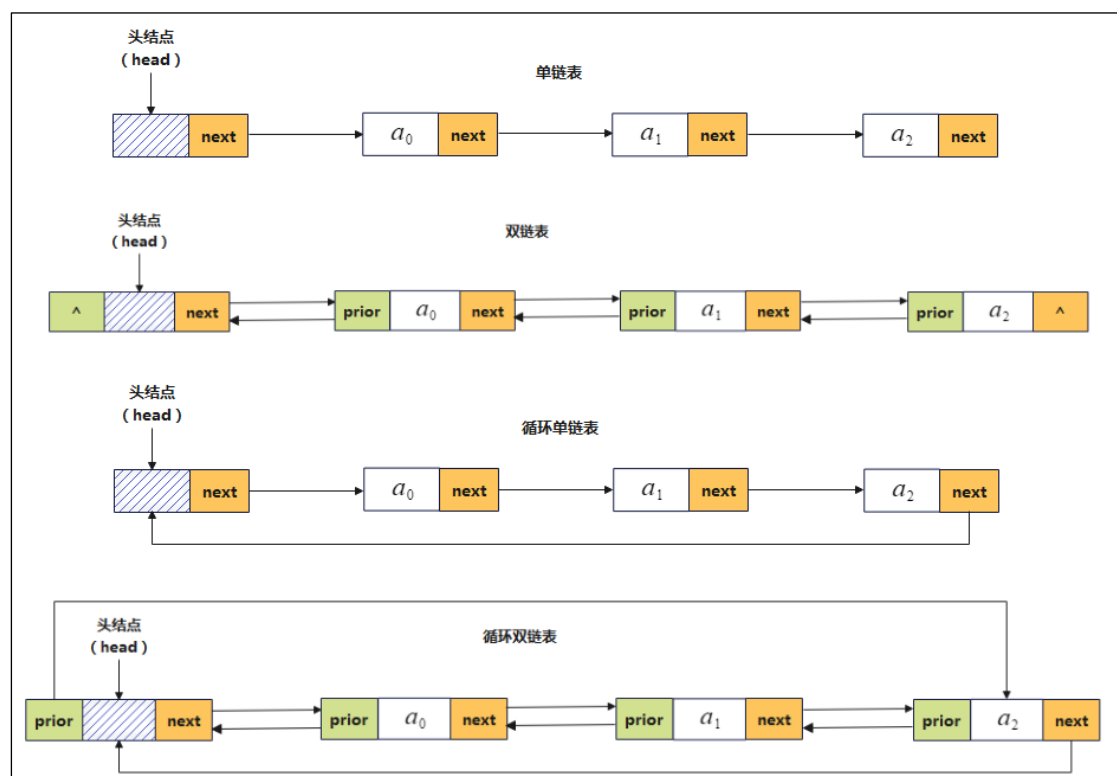
物理结构：与数组不同，链表中的结点需要自行组织，向系统申请很多**分散**在内存各处的结点，每个结点都保存了当前结点的数据和下一个结点的地址(指针)，通过指针将结点串成一串。

具体如下：



链表的分类

链表又分为单链表、双链表、循环单链表、循环双链表和静态链表。



相关概念

n 个结点离散分配，彼此通过指针相连，每个结点只有一个前驱结点，每个结点只有一个后续结点，头结点没有前驱结点，尾结点没有后续结点。确定一个链表我们只需要头指针，通过头指针就可以把整个链表都能推出来。

1.3.2 链表优缺点

1) 优点

- (1) 插入和删除操作效率高。
- (2) 动态扩展性能更好，链表不需要像数组那样预先指定固定的大小，而是可以随时动态的增长或缩小。链表是真正的动态数据结构，不需要处理固定容量的问题。

2) 缺点

- (1) 查找慢。由于链表中的结点不是连续存储的，无法像数组一样根据索引直接计算出每个结点的地址。必须从头结点开始遍历链表，直到找到目标结点，这导致了链表的随机访问效率较低。
- (2) 额外的存储空间。链表的每个结点都需要存储指向下一个结点的指针，这会占用额外的存储空间。所以，相比于数组，链表需要更多的内存空间来存储相同数量的数据元素。

1.3.3 功能定义

前文提到过链表的一些基础概念，本节我们通过实现一个链表，来进一步加深对链表的理解，需要实现以下函数：

初始化链表
<code>void initLinkedList(LinkedList *list)</code>
返回链表的长度
<code>size_t getLength(const LinkedList *list)</code>
在指定位置插入元素
<code>void insertAt(LinkedList *list, size_t index, int element)</code>
在末尾插入元素
<code>void insertEnd(LinkedList *list, int element)</code>
删除指定位置的元素并返回被删除的元素
<code>int deleteAt(LinkedList *list, size_t index)</code>
删除末尾元素
<code>int deleteEnd(LinkedList *list)</code>
获取指定位置的元素
<code>int getElementAt(const LinkedList *list, size_t index)</code>
修改指定位置的元素
<code>void modifyAt(LinkedList *list, size_t index, int newValue)</code>
释放链表内存
<code>void destroyLinkedList(LinkedList *list)</code>

1.3.4 代码实现

```
#include <stdio.h>
#include <stdlib.h>

// 链表节点结构
typedef struct Node
{
    int data;          // 节点存储的数据
    struct Node *next; // 指向下一个节点的指针
} Node;

// 链表结构
typedef struct
{
    Node *head; // 链表头节点指针
    size_t size; // 链表中的节点个数
} LinkedList;

// 初始化链表
void initLinkedList(LinkedList *list)
{
    list->head = NULL; // 初始化头节点为空
    list->size = 0;     // 初始化节点个数为0
}

// 返回链表的长度
size_t getLength(const LinkedList *list)
{
    return list->size; // 返回链表的节点个数
}

// 在指定位置插入元素
void insertAt(LinkedList *list, size_t index, int element)
{
    if (index > list->size)
    {
        return; // 忽略无效的插入位置
    }

    Node *newNode = (Node *)malloc(sizeof(Node)); // 创建新节点
    newNode->data = element;

    if (index == 0)
```

```
{
    newNode->next = list->head;
    list->head = newNode; // 新节点成为头节点
}
else
{
    Node *prevNode = list->head;

    for (size_t i = 0; i < index - 1; i++)
    {
        prevNode = prevNode->next;
    }

    newNode->next = prevNode->next;
    prevNode->next = newNode; // 新节点插入在上一节点之后
}

list->size++; // 更新节点个数
}

// 在末尾插入元素
void insertEnd(LinkedList *list, int element)
{
    insertAt(list, list->size, element); // 在链表末尾插入元素
}

// 删除指定位置的元素并返回被删除的元素
int deleteAt(LinkedList *list, size_t index)
{
    if (index >= list->size)
    {
        return -1; // 忽略无效的删除位置
    }

    int deletedElement;

    if (index == 0)
    {
        Node *temp = list->head;
        list->head = temp->next;
        deletedElement = temp->data;
        free(temp); // 释放被删除节点的内存
    }
    else
```

```
{
    Node *prevNode = list->head;

    for (size_t i = 0; i < index - 1; i++)
    {
        prevNode = prevNode->next;
    }

    Node *temp = prevNode->next;
    prevNode->next = temp->next;
    deletedElement = temp->data;
    free(temp); // 释放被删除节点的内存
}

list->size--; // 更新节点个数
return deletedElement;
}

// 删除末尾元素
int deleteEnd(LinkedList *list)
{
    return deleteAt(list, list->size - 1); // 删除链表末尾的元素
}

// 获取指定位置的元素
int getElementAt(const LinkedList *list, size_t index)
{
    if (index >= list->size)
    {
        return -1; // 返回无效的索引
    }

    Node *currentNode = list->head;
    for (size_t i = 0; i < index; i++)
    {
        currentNode = currentNode->next;
    }

    return currentNode->data; // 返回指定位置的元素
}

// 修改指定位置的元素
void modifyAt(LinkedList *list, size_t index, int newValue)
{

```

```
if (index >= list->size)
{
    return; // 忽略无效的修改位置
}

Node *currentNode = list->head;

for (size_t i = 0; i < index; i++)
{
    currentNode = currentNode->next;
}

currentNode->data = newValue; // 修改节点的值
}

// 释放链表内存
void destroyLinkedList(LinkedList *list)
{
    Node *currentNode = list->head;
    while (currentNode != NULL)
    {
        Node *temp = currentNode;
        currentNode = currentNode->next;
        free(temp); // 释放每个节点的内存
    }

    list->head = NULL; // 头节点置为空
    list->size = 0;    // 节点个数置为 0
}

int main()
{
    LinkedList myList; // 声明链表

    initLinkedList(&myList); // 初始化链表
    printf("初始化链表成功!\n");

    insertEnd(&myList, 1); // 链表尾部插入元素 1
    insertEnd(&myList, 2); // 链表尾部插入元素 2
    printf("向链表插入了 2 个元素\n");

    printf("链表长度为: %zu\n", getLength(&myList)); // 获取链表长度

    insertAt(&myList, 1, 3); // 在索引 1 处插入元素 3
```

```

printf("在索引 1 处插入元素 3\n");

printf("链表长度为: %zu\n", getLength(&myList)); // 再次获取链表长度

printf("索引 1 处的元素为: %d\n", getElementAt(&myList, 1)); // 获取索引 1 处的元素

modifyAt(&myList, 0, 4); // 修改索引 0 处的元素
printf("把索引 0 处的元素修改为 4\n");

printf("删除索引 1 处的元素,该元素值是: %d\n", deleteAt(&myList, 1));
// 删除索引 1 处的元素

destroyLinkedList(&myList); // 销毁链表
printf("链表销毁成功!\n");

return 0;
}

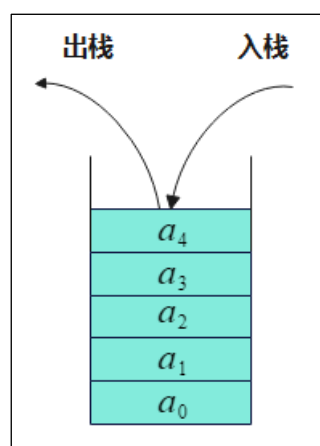
```

1.4 线性结构之栈

1.4.1 栈的定义

栈(stack), 是限制在 **只能在表的一端进行插入和删除操作** 的线性表。应用范围非常广泛。生活中也有栈的场景, 比如堆叠的盘子、报 , 电梯中的人们, 邮局的邮筒等。

特点: `后进先出` (LIFO, Last In First Out) 或 `先进后出` (FILO, First In Last Out) 的线性表。

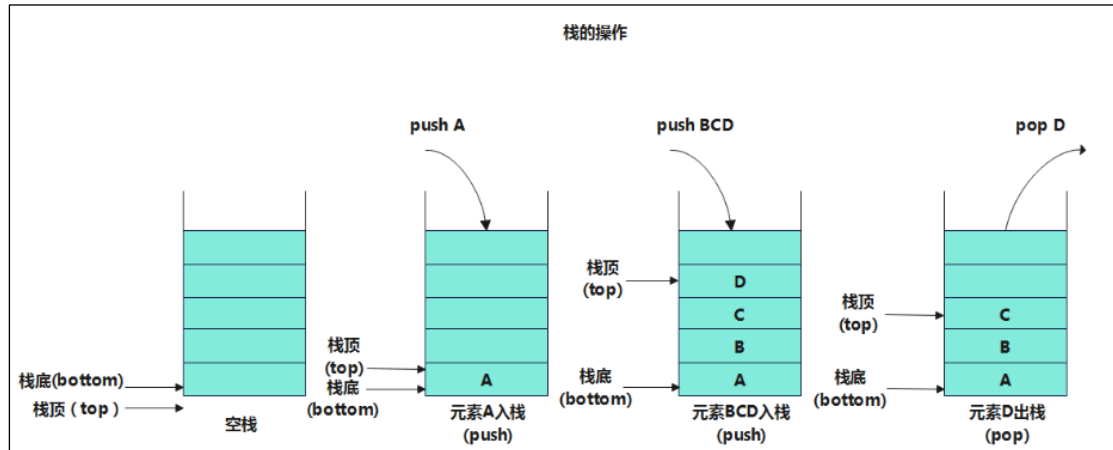


1.4.2 相关概念

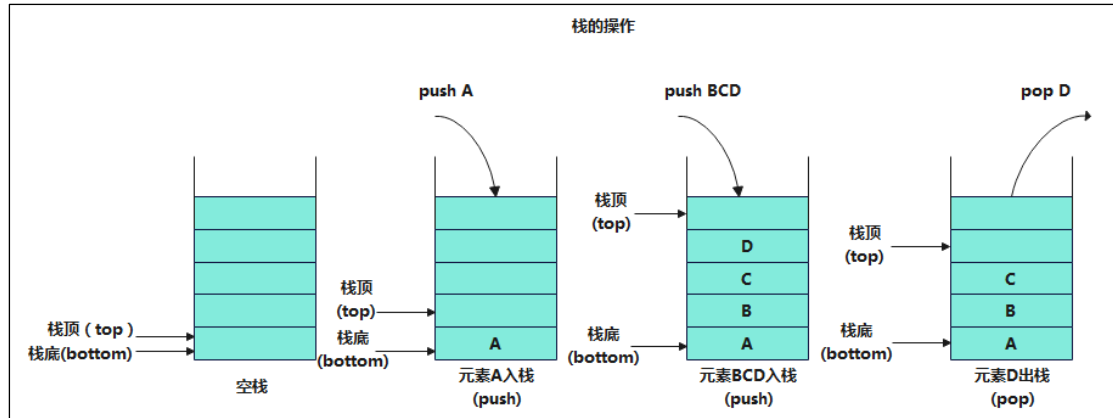
- **栈顶(Top):** 允许进行插入、删除操作的一端, 又称为`表尾`。栈顶由一个称为栈顶指针的位置指示器 (其实就是一个变量) 来指示, 它是动态变化的。

- **栈底(Bottom)**: 是固定不变的, 不允许进行插入和删除的一端, 又称为‘**表头**’。
- **空栈**: 不含任何元素的空表。
- 设栈 $S=(a_1, a_2, \dots, a_n)$, 则 a_1 称为栈底元素, a_n 为栈顶元素, 栈中元素按 $a_1, a_2, \dots, a_n$ 的次序**进栈(压栈、push)**, **出栈(弹栈, pop)**的第一个元素应为栈顶元素, 出栈顺序为: $a_n, \dots, a_2, a_1$ 。

情况 1: 初始情况 $top = -1$ 。



情况 2: 初始情况 $top = 0$ 。



1.4.3 栈的使用举例

栈最常用的地方就是**计算机的函数调用**, 不管何种语言, 最先被调用的一定最后返回。举例:

```
void funcA() {
    funcB();
}

void funcB() {
    funcC();
}
```

```
}
```

- 调用顺序是: funcA() -> funcB()-> funcC()。
- 返回顺序是: funcC() -> funcB()-> funcA()。

funcC()最后被调用，却最先被执行完，然后把结果返回给 funcB()，funcB()再执行完毕返回，把结果返回给 funcA()。

1.4.4 栈的存储结构

可用顺序表(数组)和链表来存储栈，栈可以依照存储结构分为两种：**顺序栈**和**链式栈**。

1.4.5 功能定义

前文提到过，栈的底层实现既可以使用数组也可以使用链表，本节基于动态数组实现一个栈，需要实现如下函数方法：

初始化栈

```
void initStack(Stack *stack, size_t capacity)
```

返回栈内元素个数

```
size_t getSize(const Stack *stack)
```

添加新元素

```
void push(Stack *stack, int element)
```

在末尾插入元素

```
void insertEnd(LinkedList *list, int element)
```

栈顶元素出栈并返回

```
int pop(Stack *stack)
```

释放栈内存

```
void destroyStack(Stack *stack)
```

1.4.6 代码实现

```
#include <stdio.h>
#include <stdlib.h>

// 栈结构
typedef struct
{
    int *data;        // 动态数组存储栈元素
    size_t size;      // 栈内元素个数
    size_t capacity;  // 动态数组的容量
} Stack;
```

```
// 初始化栈
void initStack(Stack *stack, size_t capacity)
{
    stack->data = (int *)malloc(capacity * sizeof(int)); // 分配初始容量的内存
    stack->size = 0; // 初始元素个数为0
    stack->capacity = capacity; // 设置容量
}

// 返回栈内元素个数
size_t getSize(const Stack *stack)
{
    return stack->size;
}

// 添加新元素
void push(Stack *stack, int element)
{
    if (stack->size == stack->capacity)
    {
        // 如果栈已满，需要扩展容量
        stack->capacity *= 2;
        stack->data = (int *)realloc(stack->data, stack->capacity * sizeof(int));
    }
    stack->data[stack->size] = element; // 将元素压入栈顶
    stack->size++; // 更新元素个数
}

// 栈顶元素出栈并返回
int pop(Stack *stack)
{
    if (stack->size == 0)
    {
        return -1; // 栈为空，返回无效值
    }
    stack->size--; // 更新元素个数
    return stack->data[stack->size]; // 返回栈顶元素
}

// 释放栈内存
void destroyStack(Stack *stack)
{

```

```

    free(stack->data); // 释放动态数组内存
    stack->data = NULL;
    stack->size = 0;
    stack->capacity = 0;
}

int main()
{
    Stack myStack;

    // 初始化栈
    initStack(&myStack, 2);
    printf("初始化栈,初始容量为 2\n");

    // 向栈压入元素
    push(&myStack, 1);
    push(&myStack, 2);

    printf("栈内元素个数:%zu\n", getSize(&myStack));

    // 弹出栈顶元素
    printf("弹出栈顶元素:%d\n", pop(&myStack));

    // 释放栈内存
    destroyStack(&myStack);
    printf("栈内存已释放\n");

    return 0;
}

```

1.5 线性结构之队列

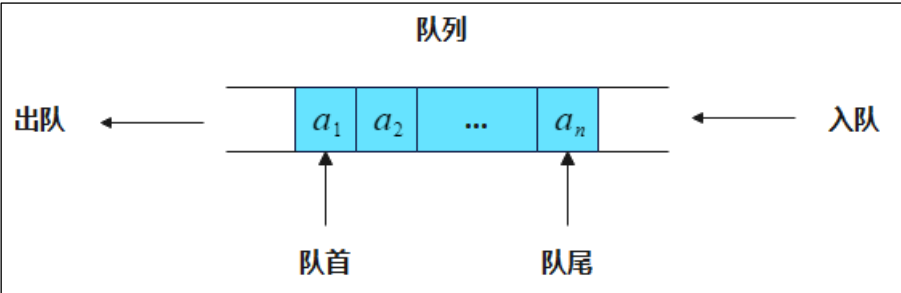
1.5.1 基本概念

队列(Queue): 也是操作受限的线性表, 限制为仅允许在表的一端进行插入(入队或进队), 在表的另一端进行删除(出队或离队)操作。

- 队首(front) : 允许进行删除的一端称为队首。
- 队尾(rear): 允许进行插入的一端称为队尾。

在空队列中依次加入元素 $a_1, a_2, \dots, a_n$ 之后, a_1 是队首元素, a_n 是队尾元素。显然退出队列的次序也只能是 $a_1, a_2, \dots, a_n$ 。

队列，是一种先进先出(First In First Out，简称 FIFO)的线性结构。类似于生活中的排队行为。



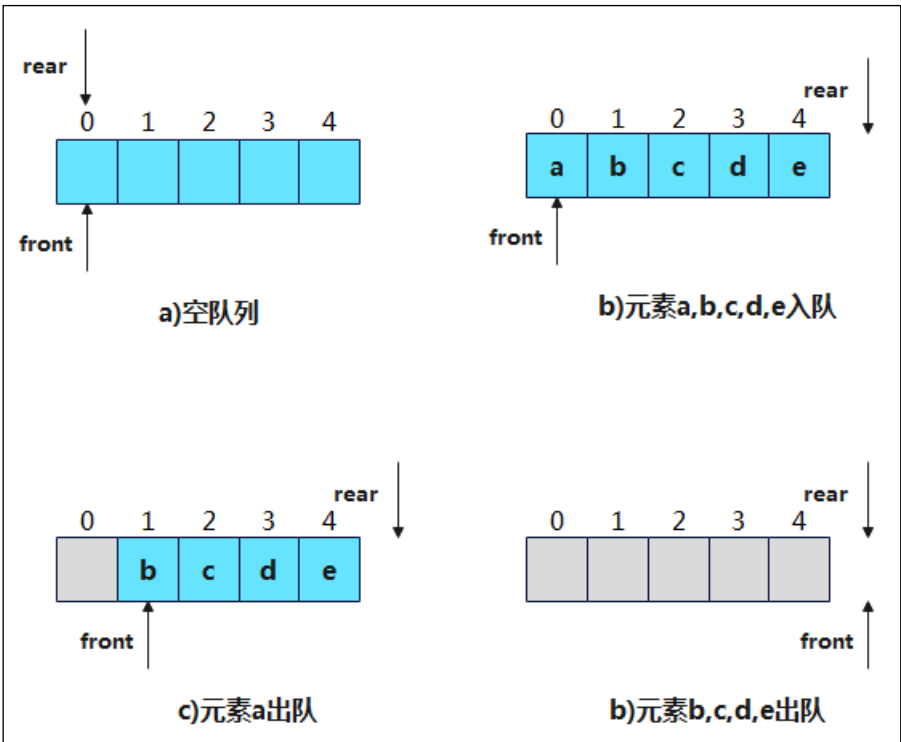
队列中没有元素时，称为空队列。

1.5.2 队列的存储结构

可用顺序表(数组)和链表来存储队列，队列按存储结构可分为顺序队列和链式队列两种。

1.5.3 队列的常见操作与问题

队列的常见操作：



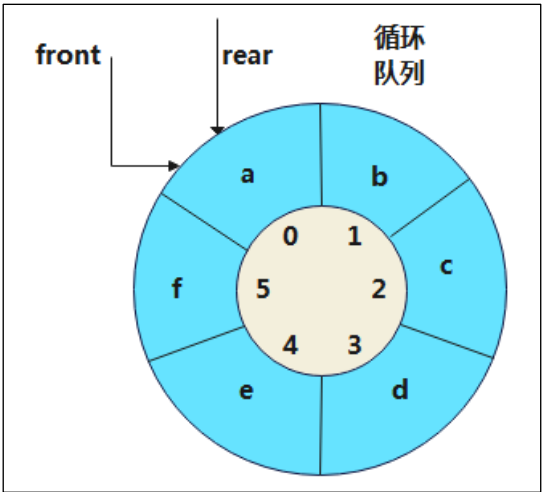
出现的问题：

顺序队列中存在“假溢出”现象：尽管队列中实际元素个数可能远远小于数组大小，但可能由于尾指针 $rear$ 已超出向量空间的上界而不能做入队操作。

为充分利用空间，克服上述“假溢出”现象，有两种方法

方法 1: 使用数组实现，入队列时添加到队列的最后，出队列时移除第一个元素同时将右侧的元素左移。

方法 2: 为队列分配的向量空间看成是一个首尾相接的圆环，这种队列称为循环队列。



1.5.4 功能定义

前文提到过，队列的底层实现也既可以选数组也可以选择链表，本节基于动态数组实现一个队列，该队列需要实现如下函数：

初始化队列
<code>void initQueue(Queue *queue, size_t capacity)</code>
返回队列内元素个数
<code>size_t getSize(const Queue *queue)</code>
添加新元素
<code>void enqueue(Queue *queue, int element)</code>
元素出队列
<code>int dequeue(Queue *queue)</code>
释放队列内存
<code>void destroyQueue(Queue *queue)</code>
遍历队列
<code>void printQueue(Queue *queue)</code>

1.5.5 代码实现

```
#include <stdio.h>
#include <stdlib.h>

// 队列结构
```

```
typedef struct
{
    int *data;        // 动态数组存储队列元素
    size_t size;      // 队列内元素个数
    size_t capacity;  // 动态数组的容量
    size_t front;     // 队列头指针
    size_t rear;      // 队列尾指针
} Queue;

// 初始化队列
void initQueue(Queue *queue, size_t capacity)
{
    queue->data = (int *)malloc(capacity * sizeof(int)); // 分配初始容量
的内存
    queue->size = 0;                                     // 初始元素个数为
0
    queue->capacity = capacity;                          // 设置容量
    queue->front = 0;                                     // 队列头指针
    queue->rear = 0;                                      // 队列尾指针
}

// 返回队列内元素个数
size_t getSize(const Queue *queue)
{
    return queue->size;
}

// 添加新元素
void enqueue(Queue *queue, int element)
{
    if (queue->size == queue->capacity)
    {
        printf("队列已满，添加失败\n");
        return;
    }
    queue->data[queue->rear] = element;                  // 将元素添加到队列尾部
    queue->rear = (queue->rear + 1) % queue->capacity; // 循环更新队列尾指针
    queue->size++;                                       // 更新元素个数
}

// 元素出队列
int dequeue(Queue *queue)
{
    if (queue->size == 0)
```

```
{
    return -1; // 队列为空，返回无效值
}
int dequeuedElement = queue->data[queue->front]; // 获取队列头部元素
queue->front = (queue->front + 1) % queue->capacity; // 循环更新队列
头指针
queue->size--; // 更新元素个数
return dequeuedElement; // 返回出队的元素
}

// 释放队列内存
void destroyQueue(Queue *queue)
{
    free(queue->data); // 释放动态数组内存
    queue->data = NULL;
    queue->size = 0;
    queue->capacity = 0;
    queue->front = 0;
    queue->rear = 0;
}

//遍历队列
void printQueue(Queue *queue){
    for(int i = queue->front, j = 0; j < queue->size ; i++, j++){
        int data = queue->data[i % queue->capacity];
        printf("%d ", data);
    }
    printf("\n");
}

int main()
{
    Queue myQueue;

    // 初始化队列
    initQueue(&myQueue, 2);
    printf("初始化队列,初始容量为 2\n");

    // 入队元素
    enqueue(&myQueue, 1);
    enqueue(&myQueue, 2);
```

```
printf("队列内元素个数:%zu\n", getSize(&myQueue));

// 出队元素
printf("出队元素:%d\n", dequeue(&myQueue));

// 释放队列内存
destroyQueue(&myQueue);
printf("队列内存已释放\n");

return 0;
}
```

第 2 章 算法

2.1 算法基础

2.1.1 算法定义

算法是一个用于解决特定问题的有限指令序列（计算机可以执行的操作）。通俗的理解就是可以解决特定问题的方法。

算法的三种表示形式：

- （1）伪代码
- （2）自然语言
- （3）流程图

算法的五个特性：

- （1）有穷性： 一个算法必须总是在执行有穷步之后结束，且每一步都在有穷时间内完成。比如，写递归函数的时候千万当心。（算法是有穷的，但执行中的程序可以是无穷的，一直在运行）
- （2）确定性： 算法中每一条指令必须有确切的含义，不存在二义性。
- （3）可行性： 算法描述的操作都可以通过已经实现的基本运算执行有限次数来实现。（不能实现的算法要他何用？）
- （4）输入： 有零个或多个输入，以刻画运算对象的初始情况。
- （5）输出： 有一个或多个输出，以反映对输入数据加工后的结果。没有输出的算法是毫无意义的。

好算法的设计目标：

- （1）正确性： 算法应能够正确地解决求解问题。这是最重要也是最基本的标准。“正确”又分为如下四个层次：
 - 程序不含语法错误。
 - 对于几组测试数据，能得出满足要求的结果。
 - 对于精心设计的、典型而苛刻的几组输入数据能得出满足要求的结果。
 - 对于一切合法的输入数据，都能得到满足要求的结果。
- （2）可读性： 算法应容易供人阅读和交流。可读性好的算法有助于对算法的理解和修改。

(3) 健壮性： 算法应具有容错处理。当输入非法或错误数据时，算法应能适当地作出反应或进行处理，而不会产生莫名其妙的输出结果。

(4) 通用性： 算法应具有一般性，即算法的处理结果对于一般的数据集合都成立。

(5) 高效率与低存储量： 效率指的是算法执行的时间；存储量需求指算法执行过程中所需要的最大存储空间。一般地，这两者与问题的规模有关。

2.1.2 算法评价标准 1：时间复杂度(效率的度量)

效率，针对的是算法执行的时间。那如何评估算法时间开销？

1) 执行时间的度量方法

算法执行时间需通过依据该算法编写的程序在计算机上运行所消耗的时间来度量。方法通常有两种：

(1) **方式 1 事后统计**：计算机内部进行执行时间和实际占用空间的统计。

(2) 问题：必须先运行依据算法编写的程序；依赖软硬件性能（计算机硬件性能、不同的编程语言），容易掩盖算法本身的优劣；没有实际价值。

(3) **方式 2 事前分析**：求出该算法的一个时间界限函数。

与此相关的因素有：

- 依据算法选用何种策略
- 问题的规模
- 数据的初始状态
- 程序设计的语言
- 编译程序所产生的机器代码的质量
- 机器执行指令的速度

这里化繁为简，抛开硬件和编程语言的影响外，可以认为一个特定算法时间度量 T 的大小，只依赖于问题的规模（通常用 n 表示），表示成是问题规模的函数 $f(n)$ 。

2) 时间复杂度的理解

时间复杂度不是执行完一段程序的总时间，而是描述为一个算法中基本操作的总次数。

算法中基本操作重复执行的次数是问题规模 n 的某个函数 $f(n)$ ，其时间量度记作 $T(n)=O(f(n))$ ，称作时间复杂度。这里的 n 称为问题的规模，如要处理的数组元素的个数为 n ，则基本操作所执行的次数是 n 的一个函数 $f(n)$ 。

大 O 渐进表示法：大 O 符号是用于描述函数渐进性的数学符号。

举例：

```

#include <stdio.h>

void PlayGame(int n) // n 为问题规模
{
    int i = 1; // 执行 1 次
    while (i <= n) // 执行 n + 1 次
    {
        printf("数据已加载%.2lf%,即将开始\n", (i + 0.0) / n * 100); // 执行 n 次
        i++; // 执行 n 次
    }

    printf("数据加载完毕，开始战斗吧！"); // 执行 1 次
}

int main()
{
    playGame(200);

    return 0;
}

```

此时， $T(n) = 1 + (n + 1) + n + n + 1 = 3n + 3$ 。

3) 如何计算时间复杂度

技巧: 通过[直接观察法](#)、[举例验证](#)等方法计算出规模 n 的函数 $f(n)$ ，然后取其增长最快的项。

练习：确定时间复杂度

```

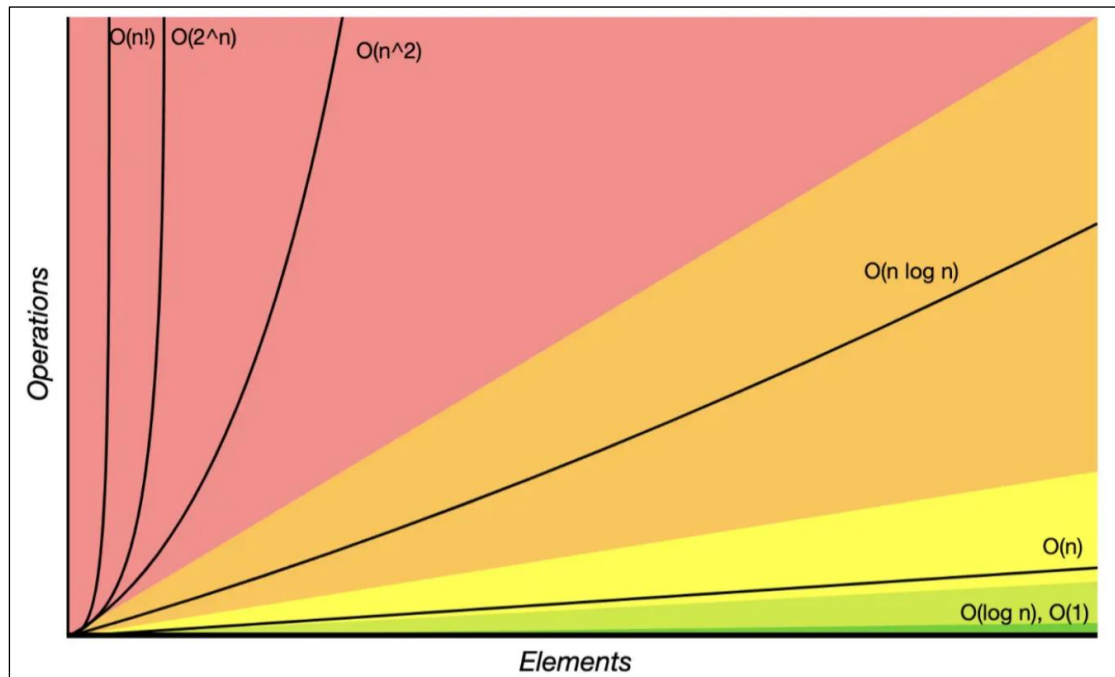
int m = 5;
int n = 10;
for (int i = 0; i < m; i++)
{
    for (int j = 0; j < n; j++)
    {
        printf("我执行了多少次呢? \n");
    }
}

```

4) 常见的时间复杂度关系

$$O(1) \leq O(\log_2(n)) \leq O(n) \leq O(n \log_2(n)) \leq O(n^2) \leq O(n^3) \leq \dots \leq O(n^k) \leq O(2^n) \leq O(n!) \leq O(n^n)$$

其中 $k > 3$ ，后三项称为指数时间复杂度，前面的各项称为多项式时间复杂度。



部分证明(了解):

当 n 趋向 $+\infty$ 时, n 比 $\log n$ 变大的速度快很多。

$$\lim_{n \rightarrow +\infty} \frac{\log_2 n}{n} \xrightarrow{\text{洛必达}} \lim_{n \rightarrow +\infty} \frac{1}{n \ln 2} = 0$$

当 n 趋向 $+\infty$ 时, 2^n 比 n^2 变大的速度快很多。

$$\lim_{n \rightarrow +\infty} \frac{n^2}{2^n} \xrightarrow{\text{洛必达}} \lim_{n \rightarrow +\infty} \frac{2n}{2^n \ln 2} \xrightarrow{\text{洛必达}} \lim_{n \rightarrow +\infty} \frac{2}{2^n \ln^2 2} = 0$$

5) 举例说明

(1) 常量阶 $O(1)$:

耗时与输入数据大小无关, 无论输入数据增大多少倍, 耗时/耗空间都不变。

常用算法或场景: 一般情况下, 一个没有循环(或有循环, 但循环次数与问题规模 n 无关) 的算法中, 原操作执行次数与问题规模 n 无关, 记作 $O(1)$, 也称常数阶。

举例:

```
void Sum1(int n)
{
    int sum = 0;           // 执行 1 次
    sum = (1 + n) * n / 2; // 执行 1 次
    printf("testSum1:%d\n", sum); // 执行 1 次
}
```

(2) 线性阶 $O(n)$:

代表数据量增大几倍，耗时也增大几倍。常用算法或场景：遍历、查找等。

举例 1:

```
void Sum2(int n){
    for (int i = 1; i <= n; i++) {
        printf("test");        //执行 n 次
    }
}
```

举例 2:

```
void Sum3(int n)
{
    int i, sum = 0; // 执行 1 次
    for (i = 1; i <= n; i++)
    {
        sum += i; // 执行 n 次
    }
    printf("testSum3:%d\n", sum); // 执行 1 次
}
```

举例 3:

```
void Search(int arr[], int n, int target) // 从 arr 数组中查找目标元素 target
{
    for (int i = 0; i < n; i++) // 从第 1 个元素开始查找
    {
        if (arr[i] == target) // 找到元素 target
        {
            printf("%d", target);
            break;
        }
    }
}
```

如果元素 target 在第一个位置：最好时间复杂度 $T(n)=O(1)$

...

如果元素 target 在最后一个位置：最坏时间复杂度 $T(n)=O(n)$

平均情况：假设元素 n 在任意一个位置的概率相同为 $1/n$ ，则平均时间复杂度为 $T(n)=O(n)$ 。因为，循环次数

$$x = (1 + 2 + 3 + \dots + n) \frac{1}{n} = \frac{n(1+n)}{2} \frac{1}{n} = \frac{1+n}{2}, \text{ 则 } T(n)=O(x)=O(n)。$$

举例 4:

```
void func(int n)
{
    int i = 1, j = 100;
```

```

while (i < n)
{
    // 基本操作
    j++;
    i += 2;
}
}

```

分析: i 的初值为 1, 每次自增 2, 假设 i 自增 m 次后循环结束, 则 i 最后的值为 $1+2m$, 因此有 $1+2m+K=n$ (其中 K 为一个常数, 因为在循环结束时 i 的值稍大于 n , 为了方便表述和进一步计算, 用 K 将 $1+2m$ 修正成 n , 因为 K 为常数, 所以这样做不会影响最终时间复杂度的计算), 解得 $m=(n-1-K)/2$, 即 $f(n)=(n-1-K)/2$, 可以发现其中增长最快的项为 $n/2$, 因此时间复杂度 $T(n)=O(n)$ 。

(3) 平方阶 $O(n^2)$:

常用算法或场景: 双层遍历、冒泡排序算法、插入排序算法等。

举例 1:

```

void Add(int x, int n)
{
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            x = x + 1;
        }
    }
}

```

举例 2:

```

void func(int n)
{
    int x = 0;
    for (int i = 0; i < n; ++i)
    {
        for (int j = i + 1; j < n; ++j)
        {
            ++x;
        }
    }
}

```

分析:

$++x$; 处于最内层循环, 因此取 $++x$; 作为基本操作。显然 n 为规模, 可以算出 $++x$; 的执行次数为 $f(n)=n(n-1)/2$, 变化最快的项为 $n^2/2$, 因此时间复杂度为 $T(n)=O(n^2)$ 。

举例 3:

```
int SumFunc(int n)
{
    int sum = 0; // 执行 1 次

    for (int i = 0; i < 100; i++)
    {
        sum += i; // 执行 100 次
    }

    for (int i = 0; i < n; i++)
    {
        sum += i; // 执行 n 次
    }

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            sum += i; // 执行 n*n 次
        }
    }

    return sum;
}
```

分析: 上述代码的时间复杂度: $1 + 100 + n + n^2$ 。大 O 表示法, 取最高阶 n^2 , 所以是 $O(n^2)$ 。

(4) 立方阶 $O(n^3)$:

举例 1:

```
{
    int sum = 0; // 执行 1 次
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            for (int k = 0; k < n; k++)
            {
                sum++; // 执行 n*n*n 次
            }
        }
    }
}
```

```

    return sum; // 执行 1 次
}

int main()
{
    int sum = test1(5);
    printf("sum = %d\n", sum); // 125 => 5^3
}

```

(5) 对数阶 $O(\log n)$:

当数据增大 n 倍时，耗时增大 $\log n$ 倍（这里的 $\log$ 是以 2 为底的，比如，当数据增大 256 倍时，耗时只增大 8 倍，是比线性还要低的时间复杂度）。

常见场景：二分查找（每找一次排除一半的可能）。

举例 1:

```

void test1(int n)
{
    int i = 1;
    while (i <= n)
    {
        i *= 2;
    }
}

```

分析：经过 m 次幂的运算可以满足 n ， $2^m = n$ ， $\Rightarrow m = \log n$ ，所以上面代码的时间复杂度： $O(1 + \log n) = O(\log n)$

(6) 线性对数阶 $O(n \log n)$:

典型代表：快速排序、归并排序、堆排序。

举例:

```

void test3(int n)
{
    int x;
    for (int i = 0; i < n; i++)
    {
        x = 1;
        while (x < n / 2)
        {
            x = 2 * x;
        }
    }
}

```

分析：执行频率最高的语句为“ $x = 2 * x$;”。该语句内层循环执行了 $\log n$ 次，外层执行了

n 次，因此总计执行次数为 $n \log n$ 次。

(7) 平方根阶 $O(\sqrt{n})$ ：

举例：

```
void fun(int n)
{
    int i = 0, s = 0;
    while (s < n)
    {
        // 基本操作
        i++;
        s = s + i;
    }
}
```

解析：假设循环执行 m 次结束，则有 $s_1=1, s_2=1+2=3, s_3=1+2+3=6, \dots, s_m=m(m+1)/2$ (其中 s_m 为执行到第 m 次时 s 的值)，则有 $m(m+1)/2+K=n$ (K 为起修正作用的常数)，由求根公式得。

$$m = \frac{-1 + \sqrt{8n + 1 - 8K}}{2}$$

即：

$$f(n) = \frac{-1 + \sqrt{8n + 1 - 8K}}{2}$$

因此复杂度为： $T(n)=O(\sqrt{n})$ 。

2.1.3 算法评价标准 2：空间复杂度(存储量的度量)

1) 空间复杂度的理解

算法的空间复杂度是指算法在运行过程中除了算法本身的指令、常数外，还需要的针对数据操作的辅助存储空间的大小。同样是问题的规模(或大小) n 的函数。

空间复杂度(Space complexity)，记作： $S(n)=O(f(n))$ 。

常见的空间复杂度主要有： $O(1)$ 和 $O(n)$ 。

2) 常见的空间复杂度关系

常见的空间复杂度关系

$$O(1) \leq O(\log_2(n)) \leq O(n) \leq O(n \log_2(n)) \leq O(n^2) \leq O(n^3) \leq \dots \leq O(n^k) \leq O(2^n) \leq O(n!) \leq O(n^n)$$

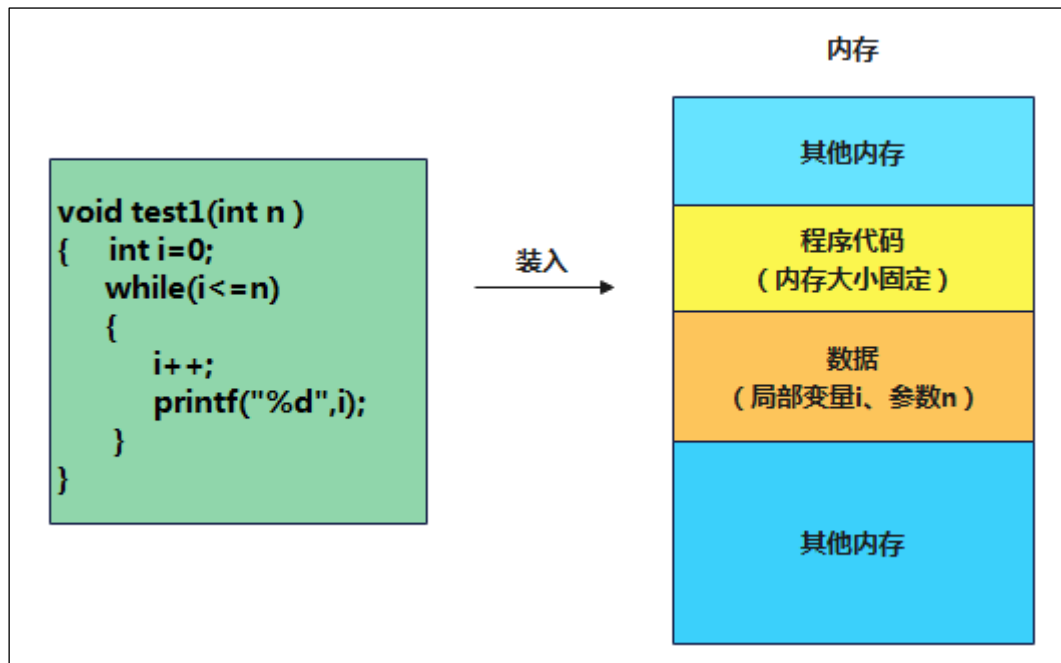
其中 $k > 3$ 。

3) 举例分析

(1) 举例 1: $O(1)$

若所需空间相对于问题规模 n 来说是常数，则称此算法为原地工作算法或就地工作算法。

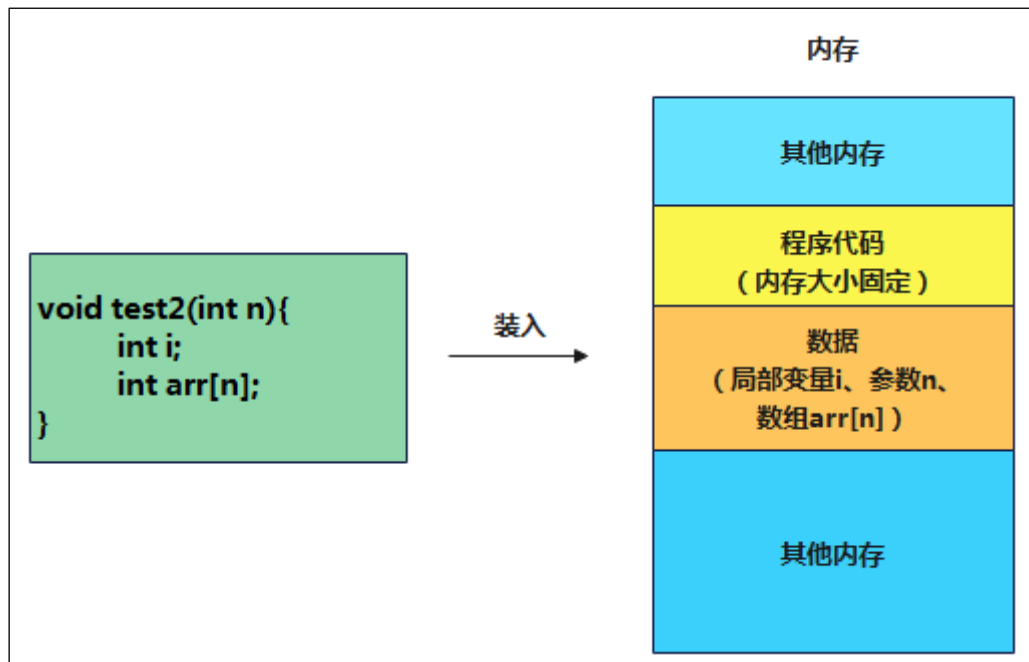
```
void test1(int n)
{
    int i = 1;
    while (i <= n)
    {
        i++;
        printf("%d", i);
    }
}
```



假设一个 `int` 变量占 4 个字节，则所需内存空间 $= 4 + 4 = 8$ ，则 $S(n) = O(1)$ 。

(2) 举例 2: $O(n)$

```
void test2(int n)
{
    int i;
    int arr[n];
}
```

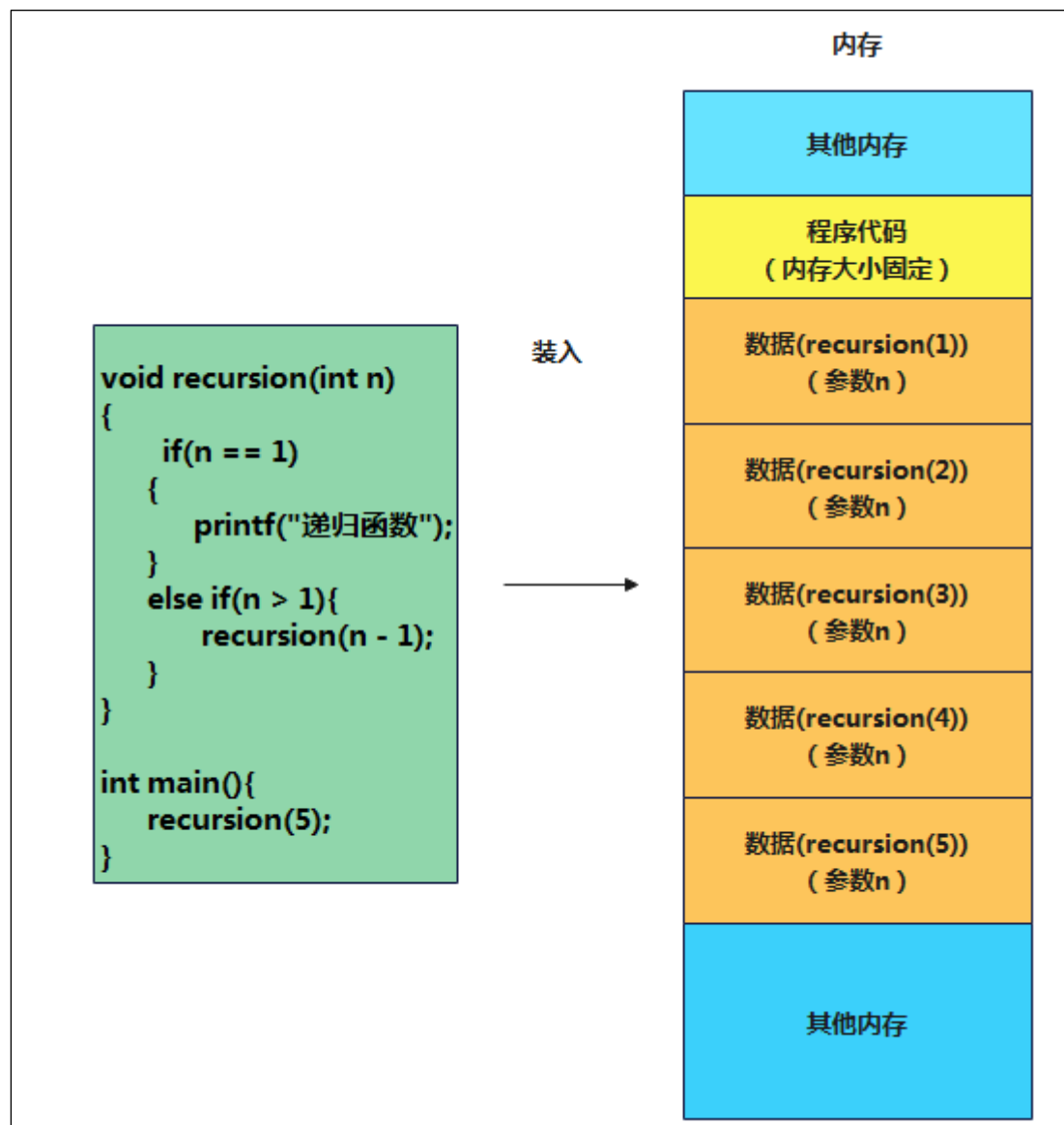


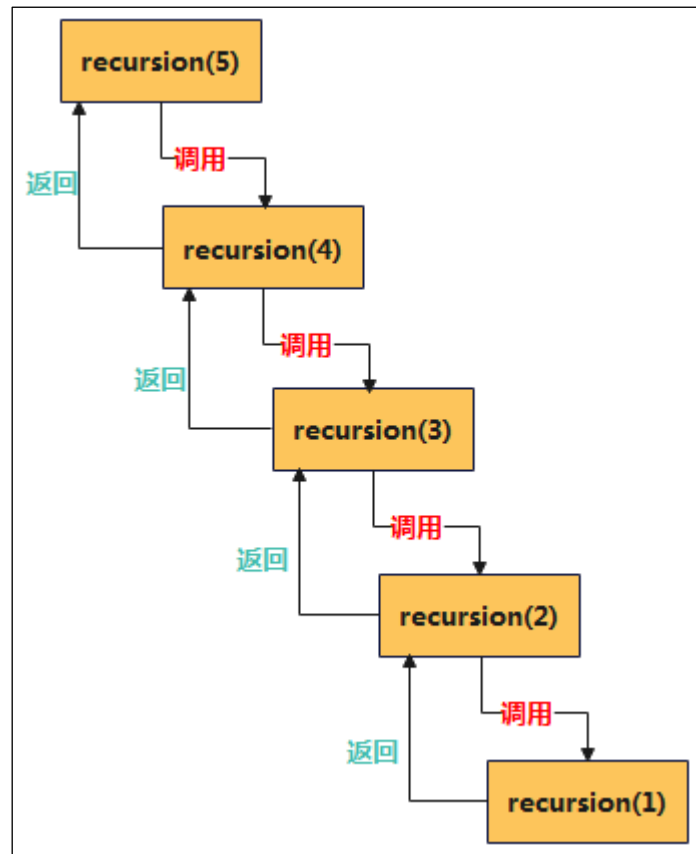
假设一个 `int` 变量占 4 个字节，则所需内存空间 $= 4 + 4n + 4 = 4n + 8$ ，则 $S(n) = O(n)$ 。

(3) 举例 3: $O(n)$

```
void Recursion(int n)
{
    if (n == 1)
    {
        printf("递归函数");
    }
    else if (n > 1)
    {
        recursion(n - 1);
    }
}

int main()
{
    recursion(5);
}
```





(4) 举例 4: $O(n^2)$

```
void test2(int n)
{
    int i;
    int arr[n][n];
}
```

2.2 查找算法

2.2.1 顺序查找

2.2.1.1 算法原理

顺序查找又称线性查找，是一种基本的查找算法，其原理是：

- (1) 从头开始遍历：从数据集的起始位置开始，逐个检查每个元素。
- (2) 比较目标：对于每个遍历到的元素，将其与目标元素进行比较。
- (3) 查找成功：如果当前元素等于目标元素，则查找成功，返回当前元素的索引。
- (4) 查找失败：如果遍历完整个数据集仍未找到目标元素，则查找失败，返回一个特殊的标识来表示未找到。

2.2.1.2 代码实现

```
#include <stdio.h>

// 顺序查找函数
int sequentialSearch(int arr[], int size, int target) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) {
            return i; // 找到目标，返回索引
        }
    }
    return -1; // 未找到目标，返回标识
}

int main() {
    int arr[] = {4, 2, 8, 6, 1, 9, 5};
    int size = sizeof(arr) / sizeof(arr[0]);
    int target = 6;

    printf("原始数组: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    int result = sequentialSearch(arr, size, target);

    if (result != -1) {
        printf("元素 %d 找到, 索引为 %d\n", target, result);
    } else {
        printf("元素 %d 未找到\n", target);
    }

    return 0;
}
```

2.2.2 二分查找

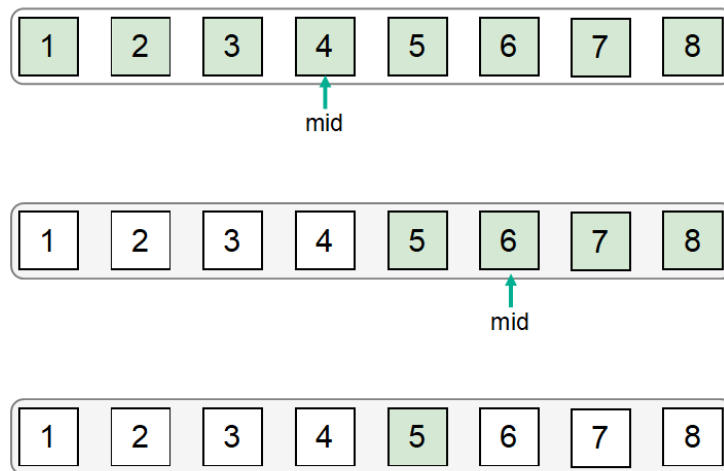
2.2.2.1 算法原理

二分查找（Binary Search）是一种高效的搜索算法，通常用于有序数据集中查找目标元素。其原理是通过将数据集划分为两半并与目标进行比较，以确定目标在哪一半中，从而逐步缩小搜索范围，直到找到目标元素或确定不存在。基本原理如下：

- （1）选择中间元素： 在有序数据集中，选择数组的中间元素。

-
- (2) 比较目标： 将中间元素与目标元素进行比较。
- (3) 查找成功： 如果中间元素等于目标元素，则查找成功，返回中间元素的索引。
- (4) 缩小搜索范围： 对于一个升序的数据集，如果中间元素大于目标元素，说明目标可能在左半部分；如果中间元素小于目标元素，说明目标可能在右半部分。根据比较结果，将搜索范围缩小到一半，继续查找。
- (5) 重复步骤： 重复上述步骤，不断将搜索范围缩小，直到找到目标元素或搜索范围为空。

查找 5



2.2.2.2 代码实现

```
#include <stdio.h>

// 二分查找函数
int binarySearch(int arr[], int size, int target) {
    int left = 0;
    int right = size - 1;

    while (left <= right) {
        int mid = (left + right) / 2;

        // 如果中间元素等于目标元素，返回中间元素的索引
        if (arr[mid] == target) {
            return mid;
        }

        // 如果中间元素大于目标元素，在左半部分继续查找
        if (arr[mid] > target) {
            right = mid - 1;
        }
    }
}
```

```
// 如果中间元素小于目标元素，在右半部分继续查找
else {
    left = mid + 1;
}
}

return -1; // 未找到目标元素，返回标识
}

int main() {
    int arr[] = {1, 3, 5, 7, 9, 11, 13, 15};
    int size = sizeof(arr) / sizeof(arr[0]);
    int target = 9;

    printf("有序数组: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    int result = binarySearch(arr, size, target);

    if (result != -1) {
        printf("元素 %d 找到, 索引为 %d\n", target, result);
    } else {
        printf("元素 %d 未找到\n", target);
    }

    return 0;
}
```

思考下：二分查找的复杂度是多少呢？ $O(\log n)$

2.3 排序算法

2.3.1 十大常见的排序算法

排序算法很多，实现方式各不相同，时间复杂度、空间复杂度、稳定性也各不相同：

排序方法	时间复杂度（平均）	时间复杂度（最坏）	时间复杂度（最好）	空间复杂度	稳定性
插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
希尔排序	$O(n\log_2 n)$	$O(n^2)$	$O(n)$	$O(1)$	不稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	不稳定
冒泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
快速排序	$O(n\log_2 n)$	$O(n^2)$	$O(n\log_2 n)$	$O(n\log_2 n)$	不稳定
归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	稳定
计数排序	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$	稳定
桶排序	$O(n+k)$	$O(n^2)$	$O(n)$	$O(n+k)$	稳定
基数排序	$O(n*k)$	$O(n*k)$	$O(n*k)$	$O(n+k)$	稳定

2.3.2 冒泡排序

2.3.2.1 算法原理

冒泡排序原理如下：

- （1）比较相邻元素：从数组的第一个元素开始，依次比较相邻的两个元素。
- （2）交换位置：如果前面的元素比后面的元素大（或小，根据升序或降序排序的要求），则交换这两个元素的位置。
- （3）一趟遍历完成后，最大（或最小）元素已移至末尾：经过一趟遍历，最大（或最小）的元素会被交换到数组的最后一个位置。
- （4）重复进行遍历和交换：除了最后一个元素，对数组中的所有元素重复执行上述两步，每次遍历都会将当前未排序部分的最大（或最小）元素放置到合适的位置。
- （5）循环遍历：重复执行步骤 3 和步骤 4，直到整个数组都被排序。



第一个元素与第二个元素比较，较大的放在后面



第二个元素与第三个元素比较，较大的放在后面



第三个元素与第四个元素比较，较大的放在后面



第四个元素与第五个元素比较，较大的放在后面



第一轮比较结束，最大的元素已经在最后面

2.3.2.2 代码实现

```
#include <stdio.h>

void bubbleSort(int arr[], int size)
{
    for (int i = 0; i < size - 1; i++)
    {
        for (int j = 0; j < size - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                // 交换 arr[j] 和 arr[j+1]
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main()
{
    int arr[] = {3, 1, 5, 4, 2};
    int size = sizeof(arr) / sizeof(arr[0]);

    printf("原始数组: ");
    for (int i = 0; i < size; i++)
    {
```

```
        printf("%d ", arr[i]);
    }
    printf("\n");

    bubbleSort(arr, size);

    printf("排序后的数组: ");
    for (int i = 0; i < size; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

2.3.3 快速排序

2.3.3.1 算法原理

快速排序（Quick Sort）由图灵奖获得者 Tony Hoare 发明，被列为 20 世纪十大算法之一，是迄今为止排序算法中速度最快的一种，快速排序的时间复杂度为 $O(n\log(n))$ 。

快速排序通常明显比同为 $O(n\log n)$ 的其他算法更快，因此常被采用，而且快排采用了分治法的思想，所以在很多笔试面试中能经常看到快排的影子。

排序思想：

从数列中挑出一个元素，称为“基准”（pivot），

重新排序数列，所有元素比基准值小的摆放在基准前面，所有元素比基准值大的摆在基准的后面（相同的数可以到任一边）。在这个分区结束之后，该基准就处于数列的中间位置。这个称为分区（partition）操作。

递归地（recursive）把小于基准值元素的子数列和大于基准值元素的子数列排序。

递归的最底部情形，是数列的大小是零或一，也就是永远都已经被排序好了。虽然一直递归下去，但是这个算法总会结束，因为在每次的迭代（iteration）中，它至少会把一个元素摆到它最后的位置去。

举例 1：

假设: [49 38 65 97 76 13 27 49]

第1趟 [27 38 13] 49 [76 97 65 49]

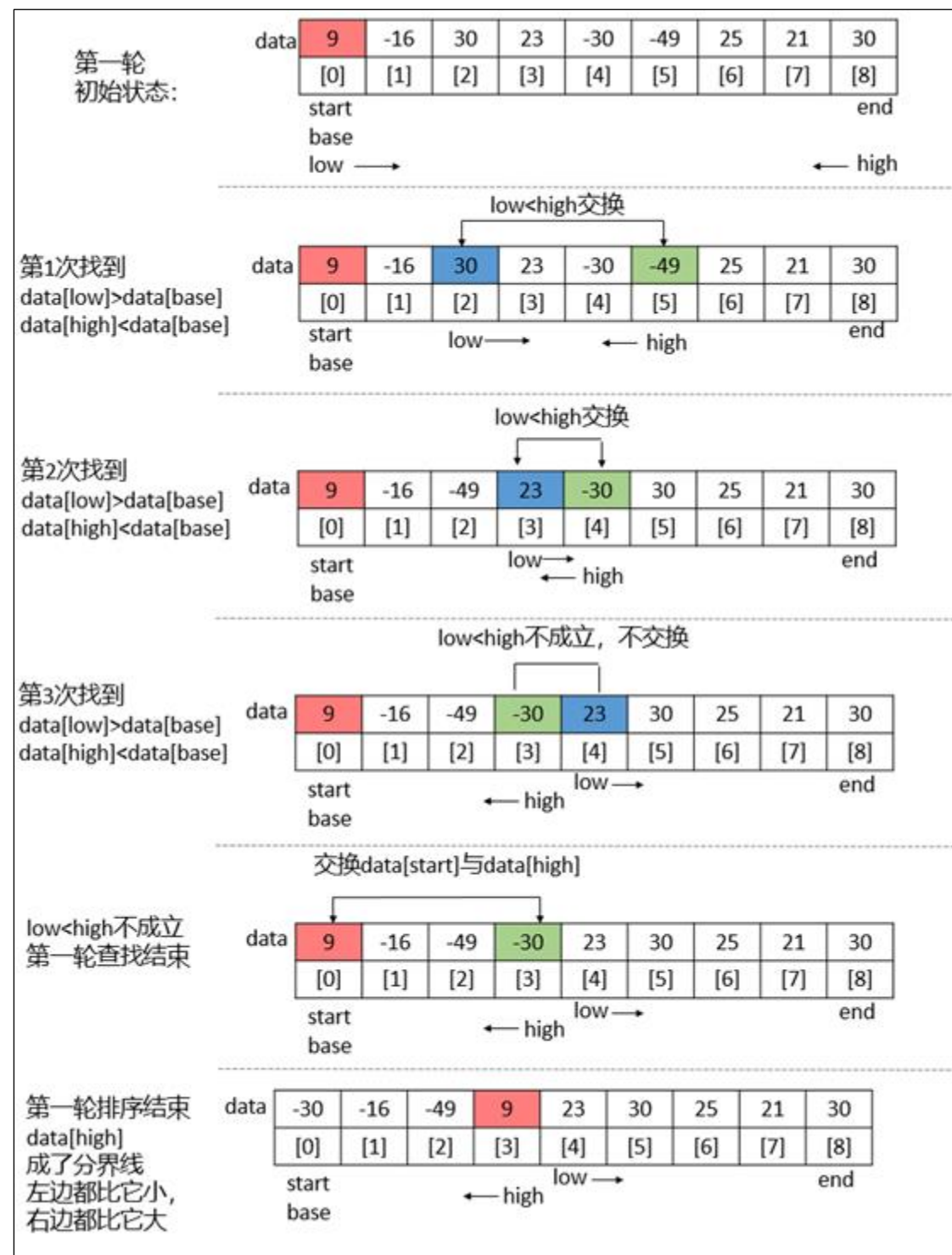
第2趟 [[13] 27 [38]] 49 [[49' 65] 76 [97]]

第3趟 [[13] 27 [38]] 49 [[49'[65]] 76 [97]]

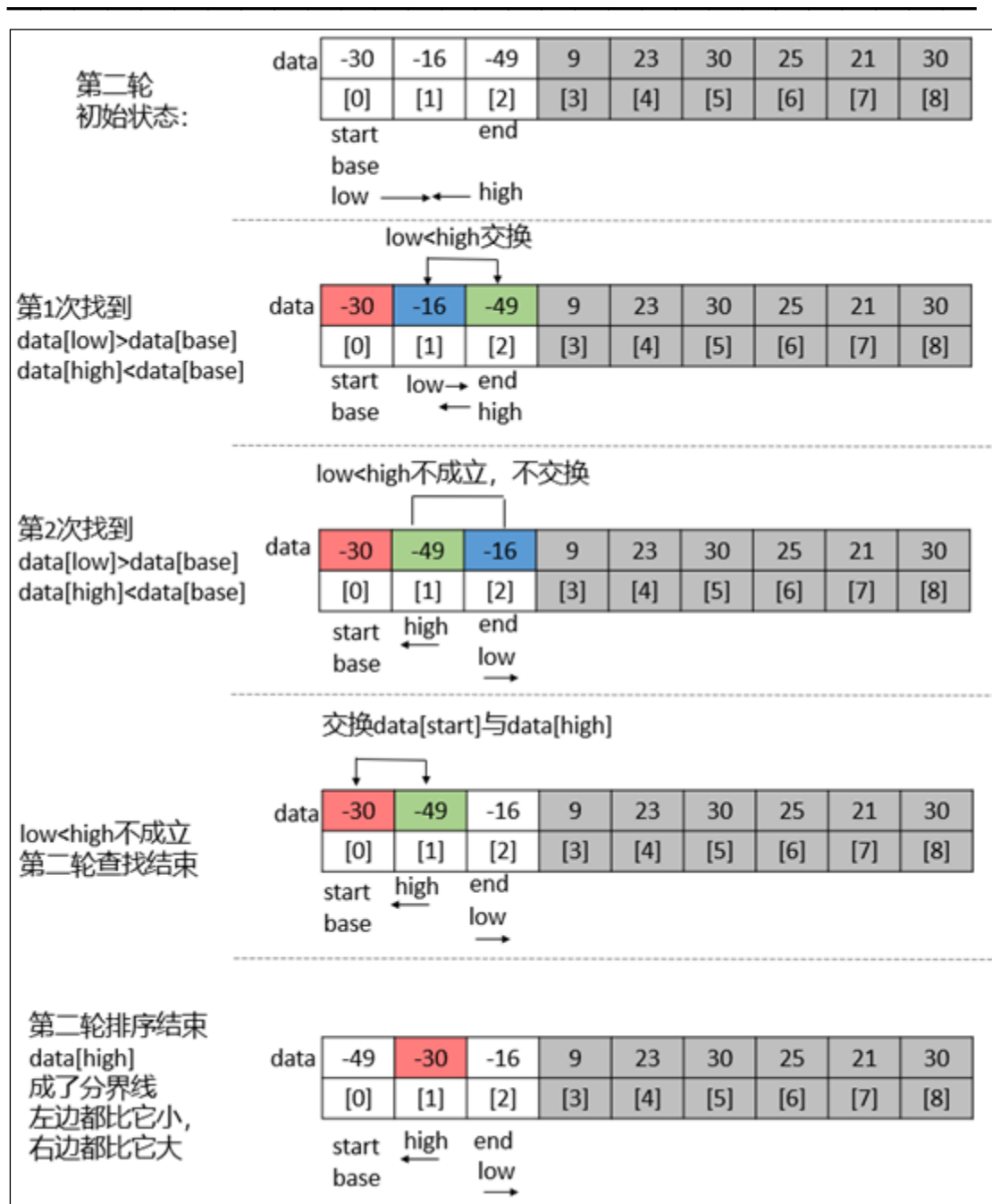
最后结果 13 27 38 49 49' 65 76 97

举例 2:

第 1 轮操作:



第2轮操作:



2.3.3.2 代码实现

```
#include <stdio.h>

void quickSort(int arr[], int size)
{
    subSort(arr, 0, size - 1);
}

void subSort(int arr[], int start, int end)
{
    if (start < end)
```

```
{
    int base = arr[start];
    int low = start;
    int high = end + 1;

    while (1)
    {
        while (low < end && arr[++low] <= base)
            ;
        while (high > start && arr[--high] >= base)
            ;

        if (low < high)
        {
            // 交换两个位置元素
            int temp = arr[low];
            arr[low] = arr[high];
            arr[high] = temp;
        }
        else
        {
            break;
        }
    }
    // 交换 high 和 start 位置的元素
    int temp1 = arr[start];
    arr[start] = arr[high];
    arr[high] = temp1;
    //继续递归调用
    subSort(arr, start, high - 1);
    subSort(arr, high + 1, end);
}
}

//打印数组
void print(int arr[], int size)
{
    for (int i = 0; i < size; i++)
    {
        printf("%d ", arr[i]);
    }

    printf("\n");
}
```

```
int main()
{
    int arr[] = {9, -16, 40, 23, -30, -49, 25, 21, 30};
    int length = sizeof(arr) / sizeof(int);

    //排序前遍历数组
    print(arr, length);

    //调用快速排序
    quickSort(arr, length);

    //排序后遍历数组
    print(arr, length);

    getchar();

    return 0;
}
```